

Automated Property-Based Fuzzing for Finding Data Manipulation Errors in Android Apps

Jingling Sun, Ting Su, Xixian Liang, Jue Wang, Geguang Pu, and Zhendong Su

Abstract—Like many software applications, *data manipulation functionalities* (DMFs) are prevalent in Android apps, which perform the common CRUD operations (*create, read, update, delete*) to handle app-specific data. Thus, ensuring the correctness of these DMFs is fundamentally important for many core app functionalities. However, the bugs related to DMFs (named as *data manipulation errors*, DMEs), especially those *non-crashing logic ones*, are prevalent but difficult to find. To this end, inspired by property-based testing, we introduce a *property-based fuzzing* approach to effectively finding DMEs in Android apps. Our key idea is that, we randomly interleave different DMFs and other possible events to explore *diverse* app states for thorough validation. Specifically, our approach characterizes DMFs in (data) model-based properties and leverages the consistency between the data model and the UI layouts as the handler to do property checking. DMFs are automatically identified by analyzing execution traces that exhibit common DMF characteristics, enabling the synthesis of DMF properties across apps. To support the application of our approach, we implemented an automated GUI testing tool, PBFDRDROID+. In evaluation, PBFDRDROID+ automatically identified 146 correct and unique DMFs in 20 real-world apps with an accuracy of 92.4%. Based on these DMFs, it further detected 32 unique and previously unknown bugs in these apps. Out of the 32 bugs, 23 are non-crashing logic bugs, and 9 are crash ones. To date, 24 have been confirmed, of which 15 have already been fixed. Many of these bugs are non-trivial and lead to different types of app failures. Our further evaluation confirms that *none* of the 23 non-crashing DMEs can be found by the state-of-the-art techniques. Overall, PBFDRDROID+ enables precise and automated discovery of non-trivial DMEs by automatically identifying DMFs and verifying their properties. We have made all the artifacts publicly available at: <https://zenodo.org/records/19951139>.

Index Terms—Property-based testing, Model-based testing, Android app testing, Non-crashing functional bugs

I. INTRODUCTION

Android apps are ubiquitous and serve many different aspects of our daily life [1]. Specifically, like many software applications, *data manipulation functionalities* (DMFs for short) are prevalent in Android apps [2], [3]. These DMFs perform the common data manipulation operations (CRUD) [4] (*i.e., create, read, update, delete*) to handle app-specific data (*e.g., creating files, reading emails, deleting posts*). Thus, ensuring the correctness of these DMFs is important because they serve as the fundamental of many core app functionalities. However,

unlike the crashing bugs targeted by many existing automated GUI testing techniques [5], [6], [7], the non-crashing logic bugs related to these DMFs are seldom tackled and may lead to frustrating consequences in real-life [8], [9], [10].

A real example. In this paper, we name the bugs which fail the normal operations of DMFs (*e.g., cannot create a file*) as *data manipulation errors* (DMEs for short). Fig. 1 shows such a DME. Amaze [11], a file management app, allows users to manipulate files or folders on Android devices. However, its DMF “Rename Folder” has a serious bug (shown in Fig. 1). Specifically, a user can create a new folder named “old-name” under the directory “/storage/emulated/0/Example/” (see Fig. 1(a)~(d)), add a file “test.txt” in this folder “old-name” (see Fig. 1(e)~(h)), switch to the up-level directory “/storage/emulated/0/” (see Fig. 1(h)~(j)), and search the created folder “old-name” under “/storage/emulated/0/” (see Fig. 1(j)~(l)). As expected, the folder “old-name” can be successfully found (see Fig. 1(l)). However, in this case, if the user renames the folder “old-name” to “new-name” (see Fig. 1(l)~(o)), the app fails as the folder name is not correctly updated to “new-name” (see Fig. 1(o)). Even worse, if the user opens the folder “old-name”, the original file “test.txt” in this folder cannot be found (see Fig. 1(p)). In this example, the failure of renaming the folder “old-name” is a DME, which involves three DMFs “Create folder”, “Search folder”, and “Rename folder”. These DMFs manipulate the app data *folder*. Note that the three DMFs work fine *independently* and the steps (“Create folder”, switch to the root directory, “Search folder” and “Rename folder”) are the *necessary* operations to manifest the DME.

Prevalence of DMEs. To investigate the prevalence of non-crashing DMEs (the main focus of this paper), we studied the 4 popular, well-maintained open-source Android apps with different categories on GitHub, *i.e.,* Amaze [11] (a file manager, 4K stars), AnkiDroid [12] (a card learning tool, 5.4K stars), K9Mail [13] (an email client, 7.1K stars) and WordPress [14] (a personal blogger, 2.7K stars), and their 245 non-crashing bugs which were reported from August 2018 to July 2021 (spanning three years). By examining the bug reports and reproducing the bugs, we find that a large portion of the non-crashing bugs (29% $\approx$ 71/245) are DMEs. It indicates that DMEs are indeed prevalent and effective bug finding techniques are in need.

Limitations of existing testing techniques. Modern automated GUI testing techniques have been widely used to help find functional bugs in Android apps [15], [16]. However, existing techniques have two major limitations in finding DMEs. First, the majority of existing testing tools [17], [18], [19], [20], [21], [22], [23], [24], [25], [26] are limited to

Manuscript created September, 2025; Jingling Sun is with School of Information and Software Engineering, University of Electronic Science and Technology of China, China. Ting Su, Xixian Liang and Geguang Pu are with Shanghai Key Laboratory of Trustworthy Computing, East China Normal University, China. Ting Su and Geguang Pu are the corresponding authors of this paper. Jue Wang is with State Key Lab for Novel Software Tech. and Dept. of Computer Sci. and Tech., Nanjing University, China. Zhendong Su is with Department of Computer Science, ETH Zurich, Switzerland.

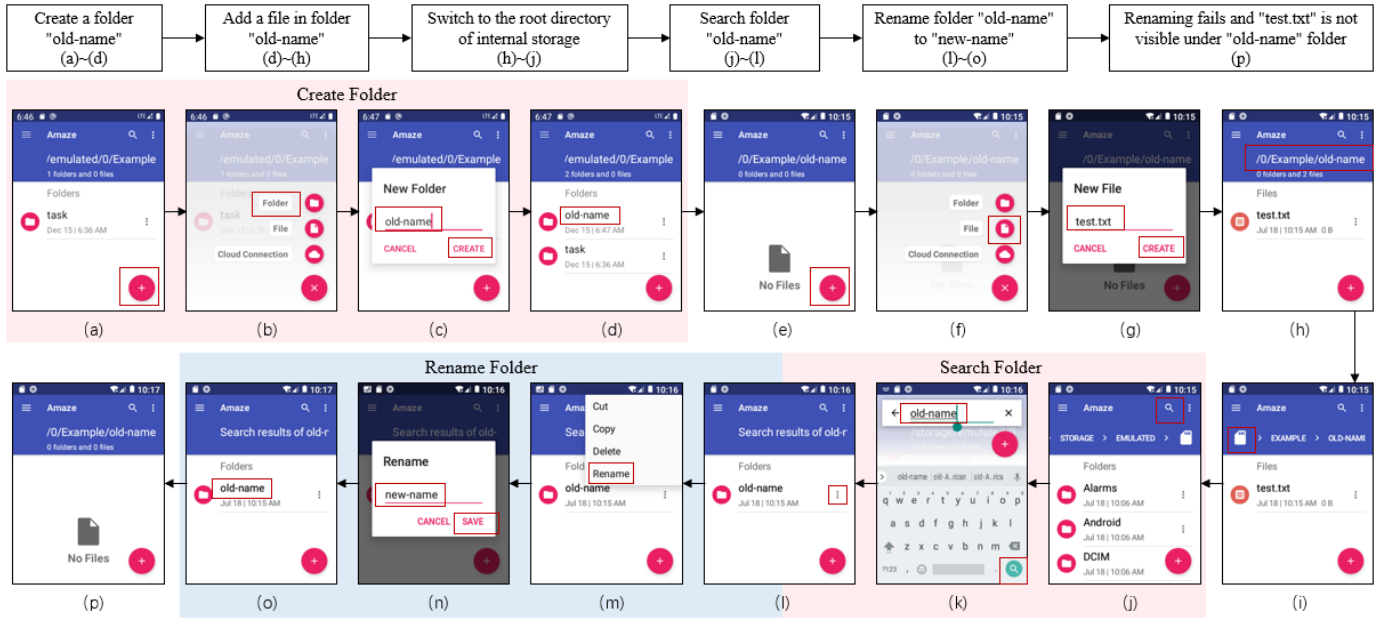


Fig. 1. A DME in Amaze (v3.7). The top graph gives an overview of this bug. The small red box on each page denotes a UI event.

crashing bugs due to the lack of strong test oracles [27]. Thus, they are difficult to find those DMEs which lead to non-crashing failures [28] (like the bug in Fig. 1). Second, although some testing tools [29], [30] can find non-crashing bugs by generating automated oracles based on heuristics [31] or metamorphic relations [32]. The oracles are too *generic* to find DMEs which are usually app-specific.

Our approach and its novelty. To this end, inspired by the classic idea of *property-based testing* (PBT) [33], we aim to introduce a novel property-based fuzzing approach to effectively finding DMEs in Android apps. Our insight is that, given a generic app functionality, we can specify its property as $\phi = \langle Pre, E, Post \rangle$, where E denotes the UI event trace performing the functionality, Pre and $Post$ denote the pre- and post-conditions before and after executing E , respectively. For example, for the DMF “Rename Folder” in Fig. 1, E denotes the event trace of “Rename Folder” (i.e., open the menu of folder “old-name”, select “Rename”, input “new-name” and click “Save” in Fig. 1(l)~(o)), Pre denotes the precondition (i.e., the menu icon of folder “old-name” on Fig. 1(l) should exist), and $Post$ denotes the postcondition (i.e., the folder name should be “new-name” on Fig. 1(o)). In this way, we can apply the classic idea of PBT to generate a number of random inputs (i.e., random UI event traces in our context) on the app under test to validate the correctness of ϕ . If there exists one random input (which leads to an erroneous program state) satisfying Pre but violating $Post$ after executing E , a bug is found. For example, the property of the DMF “Rename Folder” is falsified by the event trace in Fig. 1(l)~(o) because the folder name is not correctly updated to “new-name”.

However, applying the preceding high-level idea to fuzz DMFs is not straightforward. We face three *key* technical challenges: (1) how to characterize the property of a DMF which involves app data changes? (2) how to achieve cross-app execution and verification of DMFs despite implementation

diversity? (3) how to explore *diverse* app states to adequately validate the property of a DMF? To address these three challenges, we have three *key* observations (detailed in Section II-B): (1) the property of a DMF can be characterized by some data update effect (e.g., for the DMF “Rename Folder”, the data update effect is changing “old-name” to “new-name”) and checked by the data update effect displayed on the UI pages (e.g., in Fig. 1(o), when folder “old-name” is renamed to “new-name”, “new-name” should be displayed on the UI page), (2) the same type of DMF often exhibit some common features (e.g., a common feature of add functionalities is that the start and end pages differ solely by a newly added list entry, while all other interface elements remain consistent), even across different apps. and (3) exploring the mutual interactions between DMFs *w.r.t.* the shared app data could lead to diverse app states (e.g., in Fig. 1, the three DMFs “Create folder”, “Search folder”, and “Rename folder” *all* manipulate the shared app data “old-name” and thus manifest the DME).

Inspired by the preceding three observations, given some type of app data of interest, our idea is to (1) characterize the property of each DMF by a (data) model-based property $\phi = \langle Pre, E, R, Post \rangle$, where R records the data update effect of E in the abstract data model, thus facilitating property checking; (2) utilize the characteristics of DMFs to match the actual execution event sequences in different apps, integrating these specific execution information with property templates to synthesize app-specific properties; and (3) randomly interleave the relevant DMFs (and other possible events) to explore diverse app states, thus improving the fault detection ability. In practice, we focus on testing the DMFs in terms of five common data manipulation operations¹, i.e., *create*, *read*, *update*, *delete* and *search*.

Evaluation and results. We implemented a GUI testing tool, PBFDRDROID+, to support the application of our property-

¹In our work, we differ *read* and *search*: *read* denotes viewing existing data entries, while *search* denotes retrieving data entries with some criterion.

based fuzzing approach. We applied PBFDRDROID+ to find DMEs in 20 popular Android apps (18 open-source and 2 industrial apps). PBFDRDROID+ identified 146 different DMFs in these apps and automatically generated properties for them. Based on these properties, PBFDRDROID+ successfully found 32 unique and previously unknown bugs in 17 apps. Specifically, among the 32 bugs, 23 are non-crashing logic bugs, and 9 are crashing bugs. We reported all the bugs to the developers. To date, 24 have been confirmed and 15 have already been fixed by the app vendors. The remaining bugs are pending developer responses, with none having been rejected. Our further evaluation confirms that *none* of the 23 non-crashing DMEs can be found by the state-of-the-art testing techniques.

To sum up, this paper has made the following contributions:

- We introduce a property-based fuzzing approach to find DMEs, especially the non-crashing ones. Our approach combines the idea of property-based testing with model-based properties to achieve effective fuzzing for DMFs.
- We propose an automated property synthesis approach that leverages common DMF characteristics to identify their concrete implementations across different apps and generate properties for recognized functionalities.
- We propose a novel idea, *i.e.*, randomly interleaving different DMFs and other possible events to generate diverse app states, for improving the fuzzing effectiveness.
- We implement a GUI testing tool, PBFDRDROID+, to support the application of our approach. PBFDRDROID+ successfully found 32 unique and previously unknown bugs, 30 of which are DMEs (23 are non-crashing logic bugs). PBFDRDROID+ significantly complements existing GUI testing techniques.

In an earlier version of this work, we studied how to characterize and verify DMFs, as well as how to explore app states. In that version, DMF properties were manually defined for each app. This limited automation and required strong domain knowledge. In this paper, we address this limitation by introducing a fully automated property synthesis approach that eliminates the need for manual DMF specification. This not only removes a key bottleneck but also enables scalable testing across different apps. The core novelty of this extension lies in three aspects. First, we investigate a new problem: cross-app execution and verification of DMFs under implementation diversity. Prior work only supports per-app properties, whereas we aim at a unified property formulation that generalizes across apps. Second, we introduce a new abstraction approach that distinguishes between properties, which specify the expected behavior of a specific functionality in an app, and property characteristics, which capture the shared behavior of the same DMF across different apps. This two-level abstraction enables cross-app generalization. Third, building on this abstraction, we develop a fully automated property synthesis approach with property refinement capabilities, including redundancy elimination and precondition refinement, which enables more accurate and high-quality property construction without manual effort.

Overall, we make the following extensions to our previous work: (1) Section II-B and Section II-C2 extend the definition of abstract properties and the cross-app execution/verification

mechanism for DMFs. (2) Section III-B 2) introduces a fully automated property synthesis module that explores apps, collects GUI traces, matches them against abstract properties of predefined DMF categories, and instantiates properties using templates. In addition, Section III-D introduces two new modules, redundancy elimination and precondition refinement, which automatically remove redundant properties and complete missing preconditions, thereby reducing manual effort and improving property quality. (3) Section IV-B evaluates our approach using an updated benchmark of 20 apps, incorporating additional metrics including accuracy, redundancy, and approximate recall compared with manual identification, to better quantify the gap between automated and human analysis. In addition, Section IV-D introduces an additional evaluation of the false positive rate under fully automated usage of our tool, and compares it with state-of-the-art approaches to assess robustness in practical settings.

(4) Section V discusses two usage modes of PBFDRDROID+, namely manual refinement after property generation and fully automatic property synthesis without any manual modification. We analyze their differences in terms of accuracy and human effort, and discuss their respective advantages and limitations to guide users in selecting an appropriate usage strategy. We also include a discussion on the potential use of large language models (LLMs) for property refinement. Our results show that LLMs can accurately generate preconditions from natural language rule descriptions for this task. (5) We release our enhanced tool PBFDRDROID+, which integrates the automatic property synthesis module, to support reproducibility and future research. Due to efficiency considerations, we implement rule-based refinement in our open-sourced tool, while releasing the prompts to support future extensions.

II. APPROACH

This section presents our approach, a property-based fuzzing approach to finding DMEs in Android apps.

A. High-Level Idea

An Android app A is a UI-based, event-driven program. A UI page is represented by a *UI layout* L , which contains a number of UI views (or widgets). A *UI view* w ($w \in L$) has some attributes, *e.g.*, `className` (*i.e.*, the view type), `resourceId` (*i.e.*, the view id), `text` (*i.e.*, the text displayed by the view) and *etc.*

Definition 1. UI event. A UI view w can be executed by a UI event e . We denote e as $e = (t, w, o)$, where $e.t$ denotes the event type (*e.g.*, `click`, `edit`), $e.w$ denotes the UI view w on which e is executed, and $e.o$ denotes the optional data of e (*e.g.*, the text inputted by `edit`).

Examples. In Fig. 1, page (c) corresponds to a UI layout L . On L , an edit field view w exists (its `className` is `EditText`, and `text` is “old-name”). The event e of inputting “old-name” in w can be represented as $e = (\text{edit}, w, \text{“old-name”})$.

Definition 2. Test Inputs for Apps. An app A accepts as a test input in the form of a sequence of UI events

TABLE I
PRE- AND POST-CONDITIONS, SEMANTICS, AND DATA UPDATE RULE OF DMFs IN THE FIVE COMMON DATA MANIPULATION OPERATIONS.

op	Pre^{op}	Characteristic of E^{op} (L_0 and L_n are the UI layouts before and after E^{op} 's execution)	R^{op}	$Post^{op}$
CREATE	$e_1.w \in L_0$	$(\exists w_l, (L_n \setminus \{w_l\}) == (L_0 \setminus \{w_l\}))$ $\wedge L_n.w_l.childs - L_0.w_l.childs == 1$ $\wedge L_0.w_l.childs - L_n.w_l.childs == 0)$ $\wedge (\exists w_d \in L_{i(0 \leq i < n)}, w_d.text == L_n.w_l.childs - L_0.w_l.childs)$	$D_n := D_0 \cup \{L_i.w_d.text\}$	$\exists w, L_i.w_d.text \mapsto w \wedge w \in L_n$
READ	$e_1.w \in L_0$ $\wedge D_0 \neq \emptyset$	$(\exists w_s, e_n.w.text \mapsto w_s) \wedge w_s \in L_n \wedge n == 1$	-	$\exists w, e_n.w.text \mapsto w \wedge w \in L_n$
UPDATE	$e_1.w \in L_0$	$(\exists w_l, (L_n \setminus \{w_l\}) == (L_0 \setminus \{w_l\}))$ $\wedge L_0.w_l.childs - L_n.w_l.childs == 1$ $\wedge L_n.w_l.childs - L_0.w_l.childs == 1)$ $\wedge (\exists w_d \in L_{i(0 \leq i < n)}, w_d.text == L_n.w_l.childs - L_0.w_l.childs)$ $\wedge (\exists w_{d'} \in L_{j(0 \leq j < n)}, w_{d'}.text == L_n.w_l.childs - L_0.w_l.childs)$	$D_n := D_0 \setminus \{L_i.w_d.text\}$ $\cup \{L_j.w_{d'}.text\}$	$(\neg \exists w, L_i.w_d.text \mapsto w \wedge w \in L_n)$ $\wedge (\exists w', L_j.w_{d'}.text \mapsto w' \wedge w' \in L_n)$
DELETE	$e_1.w \in L_0$	$(\exists w_l, (L_n \setminus \{w_l\}) == (L_0 \setminus \{w_l\}))$ $\wedge L_0.w_l.childs - L_n.w_l.childs == 1$ $\wedge L_n.w_l.childs - L_0.w_l.childs == 0)$ $\wedge (\exists w_{d'} \in L_{i(0 \leq i < n)}, w_{d'}.text == L_n.w_l.childs - L_0.w_l.childs)$	$D_n := D_0 \setminus \{w_d.text\}$	$\neg \exists w, w_{d'}.text \mapsto w \wedge w \in L_n$
SEARCH	$e_1.w \in L_0$ $\wedge D_0 \neq \emptyset$	$e_1.t == edit \wedge e_n.w.resourceId == specialId$ $\wedge (\exists w, e_1.o \mapsto w \wedge w \in L_n)$	-	$e_1.o \in D_0 \implies (\exists w, e_1.o \mapsto w \wedge w \in L_n)$

(i.e., event trace). An event trace E can be denoted as $E = [e_1, \dots, e_i, \dots, e_n]$, where e_i is an event. When E is executed on A , we can obtain a sequence of the app states S , i.e., $S = [s_0, \dots, s_{i-1}, s_i, \dots, s_n]$, or denoted by $s_0 \xrightarrow{e_1} s_1 \dots s_{i-1} \xrightarrow{e_i} s_i \dots s_{n-1} \xrightarrow{e_n} s_n$, where s_i is the app state due to the execution of e_i on s_{i-1} ($1 \leq i \leq n$). We use $s_n = E(s_0)$ to denote the effect of executing E on the initial app state s_0 and reaching the ending state s_n .

Definition 3. App Property. Given some app functionality F , its app property ϕ is distilled from the specification of an app A . Specifically, ϕ is represented in the form of pre- and post-conditions, i.e., $\phi = \langle Pre, E, Post \rangle$, where E denotes an event trace corresponding to F . Each event in E is represented as a tuple $\langle action, target widget, data \rangle$, which precisely specifies the interaction with a particular UI element rather than a coarse-grained action. Pre is the precondition imposing a necessary condition that must hold before execution of E , and $Post$ is the postcondition defining the effect on the app state after executing E . Specifically, an app property ϕ can be interpreted as, if $s \models Pre$ and $s' = E(s)$, $s' \models Post$ should hold (s and s' are the states before and after the execution of E , respectively).

Property-based Testing. Property-based testing (PBT) [33] validates the correctness of a piece of program under test against some specified properties (i.e., the test oracles [27]). Specifically, it generates a large number of test inputs to check whether the properties could be violated. For example, for a function `sort` which takes as input an array `arr` and returns the sorted `arr` with its elements in the ascending order, we can specify one of its property as `arr[i] ≤ arr[j]` (where $0 \leq i \leq j \leq N - 1$, N is the length of `arr`). The idea of PBT is to generate a number of arrays with different sizes and elements to validate whether the property holds.

Our high-level idea. Our high-level idea is to leverage the idea of property-based testing to validate app properties. Given an app A and one of its app properties $\phi = \langle Pre, E, Post \rangle$, we aim to validate the correctness of ϕ . Specifically, we aim to generate different app states $s \in \mathcal{S}$, s.t. $s \models Pre$, and check whether $s' \models Post$ always holds after the execution of E .

Here, $s' = E(s)$ and $\mathcal{S}$ denote the app states of A . If $s' \models Post$ does not hold for some s , we are guaranteed to find a violation of ϕ (when ϕ is correctly defined).

B. Challenges and Observations

However, instantiating the high-level idea for validating the properties of DMFs (i.e., the ϕ in our setting) is non-trivial. We face three technical challenges: (1) how to define and check the property of the DMF which involves app data changes? (2) how to achieve cross-app execution and verification of DMFs despite implementation diversity? and (3) how to explore diverse app states to adequately validate the property of the DMF? To address these three challenges, We define some concepts in this section to illustrate our three key observations. Section II will present our approach based on the observations. **App Data, Data Type and Data Object.** Following the common Model-View-Controller (MVC) architecture pattern [34], an Android app A accepts UI events from users to manipulate app data and visualizes the data on the UI pages. We observe that an app A may contain different types of app data from the perspective of app functionalities. Each type of app data $\mathcal{T}$ is usually associated with some DMFs. These DMFs manipulate the concrete data objects of $\mathcal{T}$ to achieve some functionality. Specifically, a *data object* d (i.e., an instance of $\mathcal{T}$) is usually associated with some UI view w on some UI layout L for visualization. Here, we use $d \mapsto w$ to denote the mapping from a data object d to its corresponding view w .

Examples. In Fig. 1, page (d) displays the created folder “old-name”, which can be viewed as a data object of data type “Folder Name” (hereafter we use “Folder” for short). Specifically, the data object “old-name” is visualized by a text view w (its `text` is “old-name”, and `resourceId` is “amaze:id/firstline”) on the page (d). This relation can be denoted by “old-name” $\mapsto w$. On page (d), the current app data of type “Folder” can be recorded as $\{\text{“old-name”}\}$.

Observation 1: The property of a DMF can be characterized by some data update effect and checked by the data update effect eventually reflected on the UI pages. We observe that, for a functionally correct app, its each DMF leads to specific impacts on the app data. Therefore, we can

record these data update effects to define the properties of these DMFs. Additionally, based on the commonly adopted MVC architecture pattern adopted by Android [34], the data update effect will eventually be reflected on the app's UI pages. Therefore, we can check the property of a DMF by checking the consistency between the recorded app data and the data displayed on the UI pages.

Example. In Amaze, the execution of DMF “Create Folder” (see Fig. 1(a)~(d)) leads to the addition of a new folder to the app data, *i.e.*, adding a data object “old-name” to the app data of type “Folder”. Therefore, the data update effect of executing DMF “Create Folder” can be represented as from $\emptyset$ to $\{\text{“old-name”}\}$. After the execution of DMF “Create Folder”, the data object “old-name” is visualized on page (d), which denotes the DMF “Create Folder” achieve its functionality correctly.

Observation 2: The common features of the same DMF can be leveraged for property synthesis. Event sequences of the same type of DMF, along with the GUI interfaces traversed during execution, often exhibit similar features. These common features can be leveraged to synthesize DMF properties in different apps.

Example. Fig. 2 illustrates the process of renaming a vocabulary library in AnkiDroid (an app for vocabulary learning). This process closely resembles the folder renaming functionality in Amaze. In both cases, the interface contains a list widget before and after execution. After execution, one element in the list widget changes, while all other elements in the interface remain unchanged.

Observation 3: Exploring the mutual interactions between DMFs w.r.t. the shared app data could lead to diverse app states. We observe that, given some type of app data, its relevant DMFs can affect each other by manipulating the shared data objects, which may lead to different app states. This observation is similar to the insight of interleaving class method calls on some shared class objects to improve testing object-oriented programs [35].

Examples. In Amaze, the three DMFs, *i.e.*, “Create Folder”, “Search Folder” and “Rename Folder”, work fine independently. However, when these three DMFs interact with each other on the shared data object “old-name” (see the event trace in Fig. 1), an erroneous app state is manifested.

C. Instantiation of the High-level Idea

1) Property Definition and Verification:

Definition 4. App State. An app state is composed of two key components: (1) the UI layout L for front-end visualization, and (2) the app data $\mathcal{D}$ stored in the background (e.g., in files or database). In this way, we can represent an app state as $s = \langle L, \mathcal{D} \rangle$, where L and $\mathcal{D}$ are the UI layout and the app data at the app state s , respectively.

Inspired by the observations 1, we characterize the property of a DMF in the form of a model-based property as $\phi^{op} = \langle Pre^{op}, E^{op}, R^{op}, Post^{op} \rangle$, where op denotes one data manipulation operation, (*i.e.*, $op \in \{\text{CREATE, READ, UPDATE, DELETE, SEARCH}\}$). Here, E^{op} is the event trace performing the DMF. Pre^{op} and $Post^{op}$ are the pre- and post-conditions,

respectively. Specifically, we introduce D , an abstract data model, to record the app data of type $\mathcal{T}$ manipulated by the DMF. D is updated by R^{op} , representing the expected data state when the DMF be executed correctly. Table I lists the model-based properties of the five types of DMFs, where each row represents the property definition of one DMF type. This definition means that starting from a UI state satisfying Pre^{op} , once E^{op} is fully executed, a DME can be detected by checking $Post^{op}$, *i.e.*, comparing the actual UI state with the recorded data model D .

How to do property checking on one DMF? We take CREATE in Table I as an example to illustrate how to do property checking on one DMF. Let the event trace E^{op} of CREATE be $[e_1, \dots, e_i, \dots, e_n]$ (we will discuss in the next section how to determine E^{op} across apps from the third column of Table I). When E^{op} is executed on some app state s_0 , we can obtain a sequence of app states $S = [s_0 = \langle L_0, \mathcal{D}_0 \rangle, \dots, s_i = \langle L_i, \mathcal{D}_i \rangle, \dots, s_n = \langle L_n, \mathcal{D}_n \rangle]$ (cf. Definition 2 and 4). Note that the precondition Pre^{op} of CREATE, $e_1.w \in L_0$ (e_1 is the first event of E^{op} , and $e_1.w$ is e_1 's target UI view), decides whether CREATE is executable on state s_0 (recall that $s_0 = \langle L_0, \mathcal{D}_0 \rangle$). During the execution of CREATE, the abstract data model D is independently updated according to the semantics of E^{op} , *i.e.*, we can obtain a sequence of mirrored app data, *i.e.*, $[D_0, \dots, D_i, \dots, D_n]$. When CREATE is successfully executed (*i.e.*, all the events of E^{op} are executed), R^{op} records data update effect by adding d into $\mathcal{D}_n$ ($d \notin \mathcal{D}_0$), and the post-condition $Post^{op}$ checks the consistency between $\mathcal{D}_n$ and D_n via the UI layout L_n . Specifically, $Post^{op}$ checks whether there exists one UI view w (w is mapped from the data object d) on the ending page L_n , which displays the data object d . If w does not exist, the property is violated and a DME is found. In this way, we can leverage D to record the app data changes and achieve property checking whenever one DMF is executed. Note that for UPDATE and DELETE in Table I, if $d \notin D$, $D \setminus \{d\}$ does not delete any element in D .

2) **DMF Identification:** Unlike traditional testing that can call a function directly, GUI testing requires a sequence of GUI events to execute a functionality. The GUI events to trigger the same type of functionality may vary across apps. Therefore, to achieve cross-app execution and verification of DMFs, the specific GUI events for each DMF (*i.e.*, E^{op}) in different apps need to be identified. Inspired by the Observation 2, we identified the characteristics of event sequences E^{op} for each category of DMFs. These characteristics are documented in the third column of Table I. Based on these characteristics, our approach first generates random inputs, executes them in the target app, then monitors the traversed UI pages to detect whether there exists a continuous sequence fragment satisfy the characteristics defined for a certain DMF category. Specification, if an execution sequence $L_0 \xrightarrow{e_1} L_1 \dots L_{i-1} \xrightarrow{e_i} L_i \dots L_{n-1} \xrightarrow{e_n} L_n$ satisfies all the conditions described in the “Characteristic of E^{op} ” column for any row in Table I, it's event sequence (*i.e.*, $[e_1, e_2, \dots, e_n]$) considered the E^{op} of the property. If such a event sequence is found, we record it and consider that the corresponding DMF is completed once this event sequence has been fully executed.

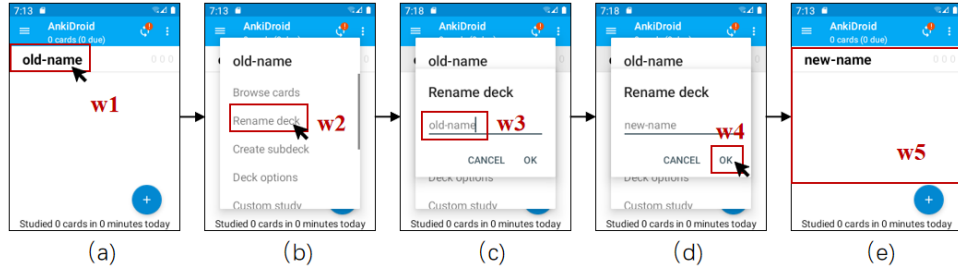


Fig. 2. The process of renaming a deck in AnkiDroid.

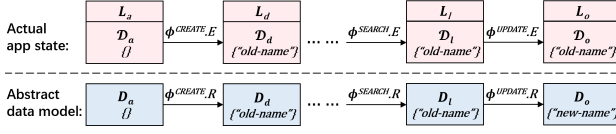


Fig. 3. Illustration of our approach on the bug in Fig. 1.

Interpretation of Event Sequence Characteristics. We take Fig.2 to illustrate the event sequence characteristics of UPDATE in Table I. As shown in Table I, the characteristic of the E^{op} for the UPDATE functionality is as follows: There exists a widget w_l (typically a list widget) such that all other layout elements remain unchanged during execution: $(L_n \setminus w_l) = (L_0 \setminus w_l)$. In Fig. 2, w_l is w_5 , with L_0 as page (a) and L_n as page (e); ignoring system widgets (e.g., battery indicator), the two pages are identical except for w_5 . The change in w_l consists of removing one existing sub-item (“old-name” here) and adding a new sub-item (“new-name” here), i.e., $|L_0.w_l.childs - L_n.w_l.childs| = 1$ and $|L_n.w_l.childs - L_0.w_l.childs| = 1$. Moreover, excluding the final interface, there are two widgets whose text matches the removed and added items, respectively (i.e., $\exists w_d \in L_j (0 \leq j < n), w_d.text == L_0.w_l.childs - L_n.w_l.childs$; $\exists w_d \in L_i (0 \leq i < n), w_d.text == L_n.w_l.childs - L_0.w_l.childs$), corresponding to w_3 on pages (c) and (d).

3) Interleaved DMF Verification: Finally, to more thoroughly verify the correctness of the DMF, it is necessary to ensure that DMFs are exercised from a wider variety of app states. Based on Observation 3, our approach randomly interleaves the relevant DMFs (with other possible events) on the shared data objects to explore diverse app state. During this exploration, our approach records the app update effects when interleaving different DMFs and performs property checking whenever the execution of a DMF is completed.

How to do property checking on multiple DMFs? We use Fig. 3 to demonstrate the process of property checking on multiple DMFs, using the example depicted in Fig. 1. Starting from the app state s_a (corresponding to page (a) in Fig. 1), the execution of the three DMFs “Create Folder”, “Search Folder” and “Rename Folder” updates the app state changes from $s_a = \langle L_a, D_a \rangle$ to $s_d = \langle L_d, D_d \rangle$, $s_l = \langle L_l, D_l \rangle$ and finally $s_o = \langle L_o, D_o \rangle$, where the L_a, L_d, L_l and L_o correspond to the UI layout of page (a), (d), (l) and (o) in Fig. 1, and D_a, D_d, D_l and D_o represent the actual app data, respectively. The abstract data model, D , is updated from $\emptyset$ to $\{\text{“old-name”}\}$, $\{\text{“old-name”}\}$ and finally $\{\text{“new-name”}\}$ according to the executions of these three DMFs. During this process, we can utilize the consistency between the recorded data in D and the UI layouts

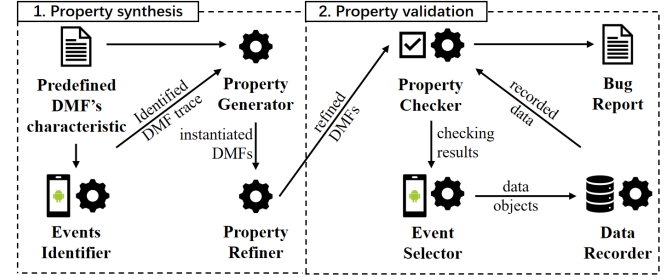


Fig. 4. Workflow of PBFDRDROID+.

to perform property checking after each DMF is executed. In this way, we identify a property violation for DMF “Rename Folder” as no widget displaying “new-name” is found on L_o .

III. IMPLEMENTATION

A. Overview

Fig. 4 shows the workflow of our property-based fuzzing technique and the implemented tool PBFDRDROID+. Specifically, PBFDRDROID+ consists of two phases. **(1) Property synthesis phase.** PBFDRDROID+ explores the target app to identify execution traces that match the characteristics of DMFs. From these traces, it extracts relevant information (the sequence of executed events and the specific data objects manipulated) and generates synthesized properties based on predefined templates. These synthesized properties are further refined to remove redundancy and improve preconditions. **(2) Property validation phase.** In this phase, the data update effect of these DMFs is recorded in the model in parallel when different DMFs are interleaved during fuzzing. And PBFDRDROID+ leverage the consistency between the recorded app data and UI pages as the handler to check the correctness of DMFs. As a result, PBFDRDROID+ can perform property testing whenever the execution of a DMF is finished.

B. Property Synthesis

To help users define DMFs more conveniently, PBFDRDROID+ includes a Property Synthesis module. This module contains a *property generator* that, given the event sequence of a DMF, its type, and its manipulated data, generates DMF definitions by instantiating predefined DMF templates. Furthermore, to improve automation, this module also contains an *event identifier* that in target apps, automatically identifies event sequences, their types, and their manipulated data objects that match the characteristics of each DMF category. In addition, we implement a *property*

refiner to automatically deduplicate and refine the synthesized properties, so that users can directly use the automatically generated properties to detect functional bugs in apps without performing any extra operations. As a result, our tool can be executed in a fully automated manner to detect DMEs. At the same time, users can also customize DMFs to test specific functionalities they care about. In the following, we first introduce how to generate testable DMF properties using recorded event sequences and templates, along with the meaning of each field in the property definition. We then present our proposed algorithm for identifying event sequences, together with their types and manipulated data objects, based on DMF characteristics. Finally, we introduce the method used to deduplicate and refine properties.

1) *Property Generator*: To support the definition of DMF properties, we design a *Property Generator* module that automatically constructs a structured and testable DMF specification based on its type, event sequences, and the data it manipulates. Fig. 5 uses the DMF “Rename Folder” to illustrate the basic procedure. As shown in Fig. 5(a), the system takes three essential inputs to define a DMF: (1) the DMF type (e.g., CREATE, READ, UPDATE, DELETE, or SEARCH), (2) the corresponding event trace, and (3) the manipulated data objects. In this case, the DMF type of “Rename Folder” is UPDATE, the event sequence is shown on L_0, L_1, L_2, L_3 (with the views of events annotated in red boxes), and its manipulated data objects are “old-name” (removed data object, on L_2) and “new-name” (added data object, on L_3). Based on the preceding inputs, this module *automatically* generates the DMF specification (defined in a JSON-style domain specific language). Fig. 5(b) shows the generated DMF specification of “Rename Folder”. Specifically, it contains six main fields: *EventTrace*, *Data*, *DataChange*, *Views*, *Precondition*, and *Postcondition*. We explain these fields as follows.

- **EventTrace**. This field records the event trace of a DMF (corresponding to E^{op} in Table I). Each event has its type, target view, and optional data. In this case, the event trace of DMF “Rename Folder” contains four events (e_1, e_2, e_3, e_4). Specifically, e_3 is an edit event, its target view is w_3 (annotated on L_2 and specified in field *Views*), and its data is *random* (which informs *input generator* to generate a random string with letters, numbers or symbols). When creating a DMF, the event trace should ensure that the pre- and the post-condition of the property could be observed on the starting page (e.g., L_0) and the ending page (e.g., L_4), respectively.
- **Data**. This field defines the manipulated data objects (corresponding to d and d' in Table I). In this case, the DMF “Rename Folder” has one removed and one added data objects, which are specified by view w_3 ’s texts on L_2 and L_3 , respectively.
- **DataChange**. This field defines the data update effect w.r.t. the DMF type (corresponding to R^{op} in Table I). In this case of “Rename Folder”, the data update effect is removing *removedDataObject* and adding *addedDataObject* in the app data D (*removedDataObject* and *addedDataObject* are specified in field *Data*).

Algorithm 1: Automatic Identification of DMFs

Inputs : A : target App, $Templates$: all property templates
Output: $DMFs$: all identified DMFs

```

1 Procedure Main:
2    $DMFs \leftarrow []$ ;  $E \leftarrow []$ ;  $L \leftarrow []$ ;
3    $L \leftarrow L :: \text{clearAndRestartApp}(A)$ ;
4   while not timeout do
5      $event \leftarrow \text{randomEvent}(\text{layout})$ ;
6      $succ, event \leftarrow \text{execute}(A, [event])$ ;
7     if succ then
8        $layout \leftarrow \text{dumpGUILayout}(A)$ ;
9        $E \leftarrow E :: event$ ;  $L \leftarrow L :: layout$ ;
10     $j \leftarrow \text{len}(E)$ ;
11    for  $i \leftarrow 0$  to  $j - 1$  do
12      foreach  $t \in Templates$  do
13         $matched, type, dataObjs \leftarrow$ 
14           $\text{checkSatisfy}(E[i : j], L[i : j], t)$ ;
15        if matched then
16           $dmf \leftarrow (E[i : j], L[i : j], type, dataObjs)$ ;
17          if  $\text{UniqueOrShorter}(DMFs, dmf)$  then
18             $DMFs \leftarrow DMFs :: dmf$ ;
19  return  $DMFs$ ;
```

- **Views**. This field defines the UI views related to the DMF. Each view can be identified by some of its attributes (e.g., *className*, *resourceId*, *text*). These views and their attributes are automatically collected according to the recorded events or user inputs.
- **Precondition and Postcondition**. These two fields define the pre- and post-conditions of a DMF (corresponding to Pre^{op} and $Post^{op}$ in Table I). The event trace of DMF can be executed only when the precondition is satisfied, while the postcondition checks whether the property of DMF is valid after the event trace of DMF is successfully executed.

Note that the mapping from data objects to the relevant views are automatically identified by this module according to the inputs. For example, given “old-name” as the removed data object, this module will analyze UI layouts ($L_0 \sim L_4$) to find the relevant views whose *text* is “old-name”. The red dotted boxes and arrows in Fig. 5 show how this module maps the removed data object (“old-name”) to the relevant view (w_5) and records this view in the DMF specification. w_5 is used in the *DataUpdate* and *Postcondition*. Moreover, since each DMF processes different data objects, the specific data object values will be abstracted from the E^{op} . The methods for abstracting E^{op} are as follows: if the input text is a data object, it is replaced with randomly generated text; if a widget attribute is a data object, attributes other than text are used for location.

2) *DMF Identifier*: To enable fully automated DMF property generation, we design an *DMF Identifier* module that automatically searches for DMFs in the target app by continuously exploring the app and checking whether execution sequences match these characteristics.

As shown in Algorithm 1, first, the algorithm initializes three lists, $DMFs$, E and L , to record all synthesized properties, all executed events, and all visited app interfaces, respectively (line 2). Next, it launches the target app and records the initial interface before starting the main loop (line 3). In the main loop (line 4-17), it continuously executes

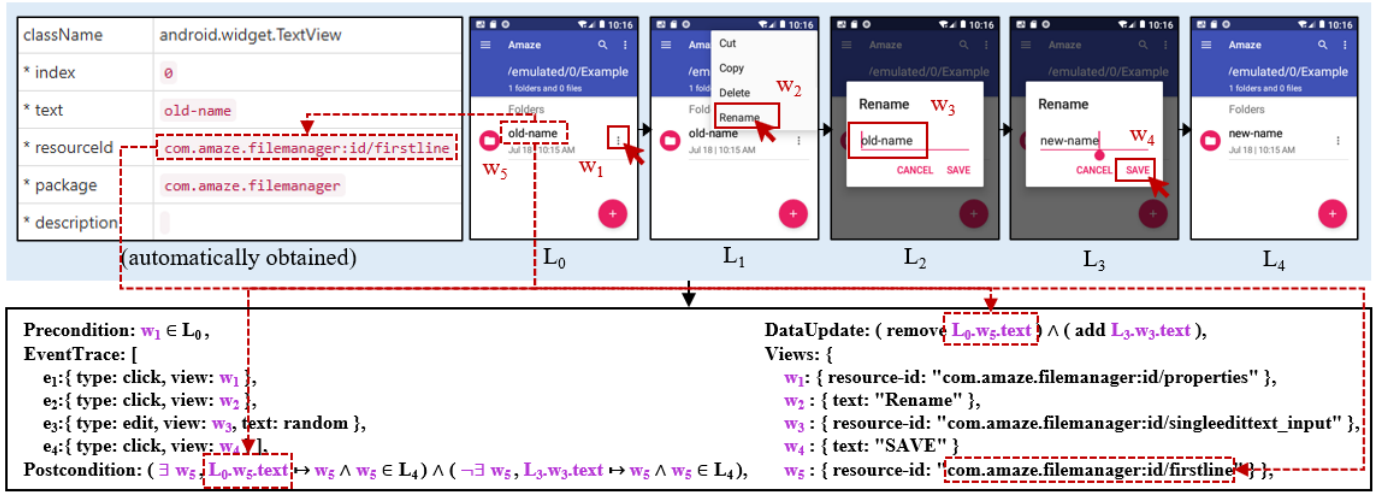


Fig. 5. The property of “Rename Folder” in *Amaze* automatically generated by PBFDR0ID+.

events and checks whether the execution sequence matches the characteristics of any DMF. Specifically, in each iteration, it randomly selects a widget from the interface and generates an interaction event for it (line 5). If the event is successfully executed (line 6-7), the interface information is updated (line 8), and both the event and the interface reached after execution are recorded (line 9). After updating the interface information, the algorithm fixes the end index j to the position of the most recently executed event, and iterates over all possible start indices i from 0 to $j - 1$ (line 10). For each subsequence $E[i : j]$ (where i and j are indices representing the start and end of the subsequence), the algorithm checks whether the sequence satisfies the characteristics of any defined DMF template by invoking the function *checkSatisfy()* (line 12-14). If a match is found, *checkSatisfy()* also leverages the matched DMF template to determine the corresponding DMF type and identify the manipulated data objects. This information, along with the matched trace, is combined into a DMF tuple $(E[i : j], L[i : j], type, dataObjs)$ (line 15). Note we convert the content of the DMF’s starting interface for removing list elements into a hash value to mark the currently identified DMF. When a DMF shares the same hash value as a historical DMF, we retain the result with the shorter event sequence (line 16-17). This approach allows us to avoid storing multiple identical DMFs and obtain a more concise DMF event sequence without the need to intentionally shorten the event sequence. Finally, the algorithm will return all DMFs found (line 18).

Example. We take the event trace in Fig. 5 as an example to illustrate how our algorithm identify the specific event sequence and generate property specification of one DMF. Assume that the execution sequence in Fig. 5 is obtained during the automatic exploration. The sequence where (a) is considered as L_0 and (d) as L_n satisfies the characteristic of UPDATE functionality. Therefore, we collect all events executed between interface (a) and interface (b) as the event sequence in the property. Then, our algorithm will abstract the E^{op} and synthesize the widgets w_d and w'_d referenced by R^{op} and $Post^{op}$. For example, based on the definition of the w_d (i.e., $w_d.text == L_n.w_l.childs - L_0.w_l.childs$)

and w'_d ($w'_d.text == L_n.w_l.childs - L_0.w_l.childs$), we determined that the added data object and deleted object processed in this execution sequence are “new-name” and “old-name”, respectively. Therefore, according to the rules for abstracting R^{op} (specific data objects should not appear in the description of event sequences), e_3 is changed to entering randomly generated text in the widget with resourceId *com.amaze.filemanager : id/singleedittext_input*. In addition, the first widget displaying “old-name” in this sequence is the widget with resourceId *com.amaze.filemanager : id/firstline* on the L_0 . This widget will be utilized as w_d in defining R^{op} and $Post^{op}$ as shown in the red dotted line of Fig. 5.

Note that our approach can be used as a fully automated method. Our experiments (shown in Section IV) demonstrate that most DMFs can be identified through the automated property generator, while the manual property identification can be used as a complementary solution in a few cases.

C. Property Validation

As illustrated in Fig. 4, the *property validation* module aims to explore diverse app states and detect property violations. The *input generator* drives the main testing loop and coordinates with the *data recorder* and *property checker* throughout the fuzzing process. The remainder of this section describes each component in turn, followed by a detailed explanation of the overall testing algorithm.

Input generator is responsible for generating and executing random GUI tests. It randomly interleaves the relevant DMFs *w.r.t.* some data type and other possible events to explore diverse app states. Note that the input generator is *compositional* like the data generators in classic property-based testing. Because it calls sub-generators to generate random string (for the text inputs of edit events), random types of events (e.g., click or long-click), and composes a sequence of events (i.e., a GUI test).

Data recorder maintains the abstract data model D according to the executed DMFs (cf. R^{op} in Table I) by *updateAppData* (used in Algorithm 2). Specifically, it *only* updates D when

Algorithm 2: Property-based Fuzzing for finding DMEs

Inputs : DMFList: app A 's relevant DMFs w.r.t. some app data type
Output: DMEs: the found DMEs (a list storing the bug-reproducing tests)

```

1 Procedure Main:
2   while not timeout do
3      $D \leftarrow \emptyset$ ;  $eventTrace \leftarrow []$ 
4     clearAndRestartApp( $A$ );
5     while  $eventTrace.len() < L_{max}$  do
6        $L \leftarrow dumpGUILayout(A)$ ;
7        $candidateDMFs \leftarrow []$ ;
8       foreach  $dmf \in DMFList$  do
9         if  $isPreconditionHold(dmf, L, D)$  then
10            $candidateDMFs.append(dmf)$ 
11       if  $rand(0, 1) > 0.5 \wedge candidateDMFs \neq \emptyset$  then
12          $dmf \leftarrow randomSelect(candidateDMFs)$ ;
13          $succ, DMFEvents \leftarrow execute(A, dmf.E)$ ;
14         if  $\neg succ$  then
15           continue;
16          $D \leftarrow updateAppData(dmf, D)$ ;
17          $L \leftarrow dumpGUILayout(A)$ ;
18         if  $\neg isPostconditionHold(dmf, L, D)$  then
19            $DMEs.append((eventTrace, DMFEvents, dmf))$ ;
20       else
21          $e \leftarrow randomEvent(L, candidateDMFs)$ ;
22          $succ, events \leftarrow execute(A, [e])$ ;
23         if  $succ$  then
24            $eventTrace \leftarrow eventTrace :: events$ ;
25   return DMEs;

```

a DMF is successfully executed. For example, only when the DMF "Rename Folder" is completely executed, the data object "old-name" will be replaced by "new-name" in D .

Property checker checks whether the precondition or postcondition of a DMF is satisfied based on the UI layout and the simulated app data (cf. Pre^{op} and $Post^{op}$ in Table I) by $isPreconditionHold$ and $isPostconditionHold$ (used in Algorithm 2). For example, when the DMF "Rename Folder" is successfully executed, this module will check whether there exists a UI view on the layout displaying the data object ("new-name"). If the UI view does not exist, the postcondition is violated.

Specifically, as shown in Algorithm 2, D is the abstract data model and $eventTrace$ records the executed events during testing. In the main testing loop (Lines 2-25), PBFDR0ID+ first initializes D and $eventTrace$, and prepares the app under test by clearing its app data (Lines 3-4), and then iteratively generates GUI tests to fuzz the app (controlled by L_{max} , the maximal length of a GUI test). PBFDR0ID+ checks which DMFs (stored in $candidateDMFs$) are qualified to be executed (Lines 7-10). Here, $isPreconditionHold$ checks whether one DMF's precondition holds according to the current app state. To randomly interleave the DMFs and other possible events, PBFDR0ID+ randomly selects one DMF when $candidateDMFs$ is not empty (Lines 12-20) or a random UI event (Line 11) by coin flipping. If one DMF is selected, all events in the event trace of this DMF ($dmf.E$ here) will be executed by $execute$ (Line 13). Note that $execute$ returns two variables: (1) $succ$ which indicates whether the DMF is

successfully executed, and (2) $DMFEvents$ which records the successfully executed events.

If the event trace of this DMF is successfully executed, the function $updateAppData$ updates the abstract data model D (Line 16). Then, function $isPostconditionHold$ checks whether the DMF's postcondition holds according to the UI layout L and the abstract app data model D (Line 18). If the property is violated, $eventTrace$ (which records all the executed events before executing the target DMF), $DMFEvents$ and dmf will be stored to reproduce the DME (Line 19). However, if the DMF fails to execute (Lines 14-15), D will not be updated, and the GUI testing will continue until reaching the limit L_{max} (Lines 5-24). And if a random event e is selected and successfully executed, this event will be recorded in $eventTrace$ (Lines 21-24). Note that function $randomEvent$ randomly selects one executable event e on the current layout L , and e should not be the first event of any DMF in $candidateDMFs$. In practice, PBFDR0ID+ uses UIAutomator [36] to execute events and obtain UI layouts.

D. Property Refiner

Although the Property Generator and DMF Identifier modules can search event sequences that match predefined property characteristics across different apps and generate executable instantiated properties, the generated properties still have two issues. First, the method may generate multiple properties that correspond to the same functionality. These properties may have slightly different event sequences or UI states before and after execution, but they represent the same behavior. Using all of them in testing will reduce the efficiency of PBFDR0ID+. Second, some properties may have imprecise preconditions, which can lead to false positives during testing. For example, in Amaze, a user can create a folder under any directory rooted at `/storage/emulated/0`". If a folder is created under `/storage/emulated/0/A`" but searched under `/storage/emulated/0/B`", the folder will not be found. This issue can be fixed by adding a more precise precondition, such as requiring the operation to start from a widget with text `/storage/emulated/0`" and resource ID `"com.amaze.filemanager:id/fullpath"`. Although such cases share common patterns and can be fixed easily, they still introduce manual effort. To further improve the quality of generated properties without any manual effort, we conduct a small-scale analysis on 10 apps. By analyzing the characteristics of all generated properties, we summarize a set of refinement rules and implement a Property Refiner module in PBFDR0ID+. This module includes two components: a de-duplication component to remove redundant properties, and a precondition refinement component to enhance incomplete preconditions. Next, we will describe the details of these two components.

Redundant Property Remover. By analyzing the characteristics of properties, we summarize two rules. These rules cover all cases observed in the 10 apps and can accurately distinguish redundant and non-redundant properties. Based on these rules, we implement a de-duplication module to remove redundant properties: First, if two properties A and B have

the same type (i.e., CREATE, DELETE, UPDATE, SEARCH, or READ), and more than half of the events in the *EventTrace* of one property are the same as those in the other, then *A* and *B* are considered redundant. PBFDRDROID+ remove the one with the longer event sequence. Second, if two properties *A* and *B* have the same type, and property *A* has the same *Postcondition* and the same effect on app data (i.e., *DataChange*) as property *B*, then they are considered redundant. PBFDRDROID+ also remove the one with the longer event sequence.

Precondition Refiner. By analyzing the characteristics of properties, we summarize two rules. These rules cover all cases observed in the 10 apps and can be used to generate more accurate preconditions. The rules are as follows. If properties for the same data type are identified multiple times (note that if this does not happen, false positives will not occur even without refinement), their starting UI states are collected. Here, the “same data type” is determined in Line 16 of Algorithm 1 based on the starting UI state and its effect on app data. For these starting UI states, PBFDRDROID+ proceed as follows.

First, common child widgets that satisfy the following conditions are searched for: the widget is not a list item, it has the same resource-id, and its `text` or `description` value may differ across UI states but must match a path pattern (match using regular expressions). If such a widget exists, a precondition is added stating that this widget must exist (defined by its resource-id and the `text` or `description` value from any one instance). For example, in Amaze, the starting UI states of CREATE folder and SEARCH folder properties both contain a widget that shows the current path. The resource-id of this widget and the path value observed when the SEARCH folder property is first identified are extracted, and they are added as preconditions to both properties. Specifically, it is required that a widget with resource-id `com.amaze.filemanager:id/fullpath` exists, and its `text` value is `/storage/emulated/0/B`. This ensures that both CREATE and SEARCH operations are performed under the same path. Second, widgets that indicate an active filter (identified by the keyword) are searched for. Such a widget does not need to appear in every starting UI state; appearing once is sufficient. If such a widget exists, a precondition is added requiring that its resource-id must not appear on the page. This rule ensures that properties are executed on the original data without filter effects. Based on the above rules, a precondition refinement module is implemented to automatically refine the generated properties.

IV. EVALUATION

Our evaluation aims to answer these research questions:

- **RQ1** : Can PBFDRDROID+ effectively identify DMFs in real-world Android apps and automatically synthesize properties for them, and how accurate is it?
- **RQ2** : Can PBFDRDROID+ find DMEs in real-world Android apps, including open-source and industrial apps?
- **RQ3** : Can the state-of-the-arts find the DMEs uncovered by PBFDRDROID+? Can PBFDRDROID+ complement them?
- **RQ4** : How do different testing strategies or configurations affect the bug finding abilities of PBFDRDROID+?

TABLE II
APP SUBJECTS EVALUATED IN OUR STUDY (K=1,000; M=100,000; B=100,000,000).

App ID	App Name	Version	#Stars	#Installations	Categories
1	Markor	2.8.6	4.5K	100K-500K	Productivity
2	Aard2	0.51	502	10K-50K	Reference
3	SimpleTask	10.9.3	557	10K-50K	Productivity
4	SkyTube	2.980	2.6K	100K-500K	Social
5	AnyMemo	10.11.7	153	100K-500K	Education
6	Amaze	3.6.7	5.7K	1M-5M	Tools
7	AnkiDroid	2.16	9.8K	10M-50M	Education
8	Wikipedia	2.7.5	2.7K	50M-100M	Reference
9	Tasks	12.5	4.4K	100K-500K	Productivity
10	RadioDroid	0.84	858	100K-500K	Music&Audio
11	ActivityDiary	1.4.2	75	1K-5K	Productivity
12	MyExpense	3.3.7	989	1M-5M	Finance
13	AndBible	5.0.8	668	500K-1M	Books
14	Antennapod	2.7.1	7.1K	1M-5M	Podcast
15	OmniNotes	6.1.0	2.8K	100K-500K	Notes
16	Commons	5.6.1	1.1K	100K-500K	Photography
17	Loop	2.3.0	8.9K	5M-10M	Habit tracking
18	Notepads	3.0.3	361	500K-1M	Productivity
19	TikTok	20.5.0	-	1B-5B	Social
20	Lark	5.14.6	-	1M-5M	Business

A. Evaluation Setup

App subjects. To our knowledge, GENIE [29] and ODIN [30] are the *only* two work which can automatically find *generic* non-crashing bugs in Android apps. They are close to our work. Thus, we selected *all* the apps from these two work as our subjects (selecting these apps also allows a fair comparison between PBFDRDROID+, GENIE and ODIN in RQ2). Specifically, GENIE and ODIN evaluated 12 and 17 apps, respectively. After removing the duplicated apps, we obtained 18 apps. From the 18 apps, we excluded 4: UnitConverter and Fosdem (as their functionalities are too simple to include any DMF), as well as Materialistic and Transistor (which have not been maintained for over two years and are no longer available on major app marketplaces). To complement the subjects, we additionally selected 4 popular open-source apps (OmniNotes, Commons, Loop and Notepad) from [37], and 2 industrial closed-source apps (TikTok [38], Lark [39]) from ByteDance. Thus, we have 20 apps in total, all of which are released on Google Play [40]. In Table II, “Version”, “#Stars”, “#Installations” and “App Feature” give the latest app versions at the time of our study, the numbers of stars on GitHub, the installations on Google Play, and the main app feature, respectively. These apps are representative with diverse features and most of them are popular. Note that our approach is not limited to specific app types.

Evaluation setup of RQ1. To evaluate whether PBFDRDROID+ can effectively identify DMFs and synthesize properties in real-world Android apps, we conducted experiments on our selected subjects. Specifically, we ran the 18 open-source apps on Android emulators (Pixel XL, Android 8.0) deployed on a 64-bit Ubuntu 18.04 machine (64 cores, AMD 2990WX CPU, and 64GB RAM), and the 2 industrial apps on the real Android device (OPPO A11s, Android 10.0). We allocated 12 machine hours for PBFDRDROID+ to extensively explore target apps, find DMFs and synthesize properties for them. To manage the exploration depth and ensure test reproducibility, we configured the length of each generated test case to 1,000 events. At the commencement of each test case, PBFDRDROID+ automatically clears the app’s information and restarts it to

maintain state isolation. It is worth noting that, PBFDRDROID+ searched for all five types of DMFs simultaneously within the app each time. After the property generation concluded, to evaluate the precision of the synthesized properties, we manually inspected each property and recorded the number of properties found by the tool, the number of correctly identified properties, the number of unique properties after manual verification, the number of non-redundant properties that were mistakenly removed during deduplication, and the number of properties that still required additional preconditions after refinement. Furthermore, to compare the performance between automated and manual property generation, we compared the automatically identified properties with all properties we manually identified in the previous conference version, recording which properties were only found by the automated synthesis method and which were only identified manually.

Evaluation setup of RQ2. We tested the same app subjects in the same environment using the properties generated in RQ1. We allocated 48 machine hours for extensively testing each app, and configured the maximum length of one GUI test generated by PBFDRDROID+ as 100 events. For each app, we manually inspected all the reported bugs to find the distinct ones which have different bug-triggering tests and manifestations. Then, we reported each distinct bug to the app vendors. Each bug is provided with a bug-reproducing test and video to ease confirmation and diagnosis. If a bug is not marked as duplicated by the app vendors, we regard it as a unique bug.

Evaluation setup of RQ3. Two major categories of automated GUI testing tools for Android exist based on the oracle problem [27]. One category of tools [29], [37], [41], [42], [30], [43] can automatically find non-crashing bugs. In this category, GENIE [29] and ODIN [30] are the only two tools which can find *generic* non-crashing bugs, which may find DMEs. Thus, we selected GENIE and ODIN for comparison. We did not select other tools because they target other specific types of non-crashing bugs rather than DMEs. We will discuss the differences between these work and ours in Section VI. The other category of tools (*e.g.*, MONKEY [17]) is limited to crashing bugs due to the lack of strong test oracles [27]. Although PBFDRDROID+ focuses on finding non-crashing bugs, we still selected MONKEY [17] for comparison. Specifically, we allocated the same 48 hours for GENIE, ODIN and MONKEY to test each of the 18 open-source apps in the same experimental environment. GENIE and ODIN were setup with their default settings in their original papers [29], [30]. To ensure a fair comparison, we operated PBFDRDROID+ in a fully automated mode. Specifically, PBFDRDROID+ automatically synthesizes properties for the apps under test based on generic templates. Once these properties are generated, we deploy them directly to detect bugs without any manual modifications or refinements, regardless of whether they are entirely correct or lack specific preconditions. The testing duration for PBFDRDROID+ (including both property synthesis and bug detection) was set to 48 hours, consistent with the time allocated to Genie, Odin, and Monkey. Following the testing phase, we manually inspected all the bugs reported by PBFDRDROID+, Genie, Odin, and Monkey to determine their

false positive rates and the number of real DMEs identified. We further analyzed which DMEs were uniquely identified by a specific tool.

Evaluation setup of RQ4. PBFDRDROID+ interleaves different DMFs and other possible events to generate diverse program states. Thus, we investigated whether different interleaving strategies could affect the bug finding abilities of PBFDRDROID+. Specifically, we setup two baseline strategies for comparison:

- **Baseline A.** Baseline A only tests *one single DMF without* interleaving with other possible events. Specifically, given one DMF, we manually identify the shortest event trace from the app starting page to the GUI page satisfying the precondition of the DMF (*e.g.*, Fig. 1(a)~(d) is a qualified event trace for the DMF “Rename Folder”). During testing, Baseline A always (1) follows this event trace to reach this DMF, (2) executes the events of the DMF, (3) randomly generates necessary text inputs for some events (*e.g.*, Edit) of the DMF, and (4) checks the postcondition.
- **Baseline B.** Baseline B interleaves *one single DMF with other possible events*. It corresponds to Algorithm 2 when the input `DMFList` only contains one DMF under test.

We conduct experiments on the 18 open-source apps, allocating the same testing time (48 hours per DMF) and environment for Baseline A, Baseline B, and PBFDRDROID+. Baselines A and B will not terminate before reaching the time bound because they are set up for running continuously.

Additionally, our experiment sets the default maximum length of each GUI test L_{max} (see Algorithm 2, Line 5) as 100 events. L_{max} constrains the possible interleavings between DMFs and other possible events, and thus may affect the tool's effectiveness. Thus, we evaluated the other two configurations of L_{max} , *i.e.*, 200 and 400 events per test, respectively, with the same testing time (48 hours per app) and experimental environment to test the 18 open-source apps, and compared the numbers of found bugs. Roughly, 40K GUI events can be generated in the 48-hour testing time for each app.

B. Results for RQ1

Table III presents the results of PBFDRDROID+ in identifying DMFs and synthesizing properties. To evaluate the quality of the synthesized properties, we manually inspect each result and record several key metrics: the total number of properties found by the tool (#FD), the number of properties remaining after automated filtering (#AF), the number of incorrectly identified properties (#FI), the number of properties that are still redundant after automated filtering (#RR), the number of non-redundant properties that are mistakenly removed during automated deduplication (#MD), and the number of properties that still require additional preconditions after refinement (#MP). Furthermore, the following six columns provide a comparative analysis between automated and manual property generation for the total DMF properties and each DMF type (*i.e.*, CREATE, READ, SEARCH, UPDATE, and DELETE). Each of these six columns is further divided into four sub-columns: (1) the number of properties automatically identified by PBFDRDROID+ (A), (2) the number of properties manually

TABLE III
EVALUATION RESULTS OF PBFDRDROID+ ON DMF IDENTIFICATION AND PROPERTY SYNTHESIS.

App Name	#FD	#AF	#FI	#RR	#MD	#MP	Total Correct				Create				Read				Search				Update				Delete			
							A	M	AO	MO	A	M	AO	MO	A	M	AO	MO	A	M	AO	MO	A	M	AO	MO	A	M	AO	MO
Markov	14	6	0	0	0	0	6	5	1	0	1	1	0	0	1	1	0	0	2	1	1	0	1	1	0	0	1	1	0	0
Aard2	3	2	0	0	0	0	2	4	0	2	1	1	0	0	0	1	0	1	1	1	0	0	0	0	0	0	0	1	0	1
Simpletask	71	10	1	0	0	1	9	4	5	0	2	1	1	0	3	1	2	0	2	1	1	0	0	0	0	0	2	1	1	0
Skytube	4	4	1	0	0	0	3	5	1	2	1	1	0	0	0	1	0	0	1	1	0	0	0	1	0	1	1	1	0	0
AnyMemo	27	10	1	1	0	0	9	7	3	1	3	2	1	0	1	1	0	0	1	0	1	0	2	2	1	1	2	2	0	0
Amaze	23	8	0	1	0	0	7	5	2	0	2	1	1	0	1	1	0	0	1	1	0	0	1	1	0	0	2	1	1	0
AnkiDroid	65	12	1	0	0	0	11	7	5	1	1	1	0	0	8	3	5	0	0	1	0	1	1	1	0	0	1	1	0	0
Wikipedia	21	7	0	0	1	0	7	5	2	0	1	1	0	0	2	1	1	0	1	1	0	0	2	1	1	0	1	1	0	0
Tasks	66	16	1	1	0	1	14	5	9	0	2	1	1	0	6	1	5	0	2	1	1	0	2	1	1	0	2	1	1	0
RadioDroid	123	18	1	1	0	0	16	5	11	0	4	2	2	0	4	1	3	0	3	1	2	0	1	0	1	0	4	1	3	0
ActivityDiary	13	6	0	0	0	1	6	5	1	0	1	1	0	0	1	1	0	0	2	1	1	0	1	1	0	0	1	1	0	0
MyExpense	24	13	1	1	0	0	11	5	7	1	2	1	1	0	6	1	5	0	1	1	1	1	1	1	0	0	1	1	0	0
AndBible	1	1	0	0	0	0	1	-	-	-	0	-	-	-	0	-	-	-	0	-	-	-	1	-	-	-	0	-	-	-
Anntennapod	17	7	1	0	0	0	6	5	1	0	1	1	0	0	1	1	0	0	2	1	1	0	1	1	0	0	1	1	0	0
OmniNotes	19	9	0	0	0	0	9	4	5	0	1	1	0	0	2	1	1	0	1	1	0	0	2	1	1	0	3	1	2	0
Commons	5	3	1	0	0	0	2	-	-	-	0	-	-	-	0	-	-	-	1	-	-	-	0	-	-	-	1	-	-	-
Loop	13	5	0	0	0	0	5	-	-	-	1	-	-	-	1	-	-	-	0	-	-	-	2	-	-	-	1	-	-	-
Notepad	0	0	0	0	0	0	0	5	0	5	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
TikTok	11	11	3	0	0	1	8	5	3	0	1	1	0	0	3	1	2	0	2	1	1	0	2	1	1	0	0	0	0	0
Lark	17	10	0	0	0	0	10	5	5	0	3	1	2	0	1	1	0	0	2	1	1	0	1	1	0	0	2	1	1	0
Sum	537	158	12	5	1	4	142	86	61	12	28	19	9	1	41	19	24	3	25	16	11	3	21	15	6	3	26	17	9	2

Notes: #FD: Total properties found; #AF: Properties remaining after automated filtering; #FI: Incorrectly identified properties; #RR: Redundant properties after filtering; #MD: Non-redundant properties mistakenly removed; #MP: Properties requiring additional preconditions after refinement. The last six columns report the number of properties identified by Automated (A) and Manual (M) methods, where AO denotes properties identified only by PBFDRDROID+, and MO denotes those identified only manually.

identified in the conference version (M), (3) the number of properties identified only by PBFDRDROID+ (AO), and (4) the number of properties identified only manually (MO). By analyzing these metrics, We can quantitatively evaluate the approximate recall of PBFDRDROID+ relative to human expertise, as well as the unique advantages of PBFDRDROID+ in discovering complex or hidden properties that might be overlooked by human reviewers.

In our experiments, the tool reported a total of 537 DMFs, which was reduced to 158 DMFs after automatically de-duplication. Through manual inspection of all reported DMFs, we found that 146 DMFs were correctly identified, achieving an accuracy of 92.4% ($\approx 146/158$). Furthermore, we observed that our refinement module is highly accurate. Specifically, only one property was incorrectly de-duplicated (due to two search functions in an app having identical execution steps and data effects/post-conditions, despite originating from different interfaces). Among the de-duplicated DMFs, only 5 remained redundant, and only 4 DMFs processed by our automatic refinement still required further manual refinement (typically caused by unique pages within the apps). Additionally, PBFDRDROID+ demonstrated its capability to identify various types of DMFs. Within the 146 correctly identified DMFs, there were 28 CREATE functions, 41 READ functions, 25 SEARCH functions, 21 UPDATE functions, and 26 DELETE functions. Finally, compared to the manually identified DMFs from our previous version, PBFDRDROID+ failed to recognize only 12 manually identified DMFs, yielding a recall of 86.0% ($\approx 74/86$), while successfully discovering 61 additional new DMFs, demonstrating the effectiveness of PBFDRDROID+.

1) *Analysis of Incorrect DMF Identifications*: We analyzed all incorrectly identified DMFs and categorized the causes of these misidentifications into three main types. (1) *Functionally similar UI behaviors (5/12 cases)*. Some non-DMF functions may produce effects similar to those of DMFs. For example, applying a filter might remove a list item from view, which could be mistakenly interpreted as a delete operation.

(2) *Partial list visibility (4/12 cases)*. Incorrect identifications can also occur when the UI fails to display all list items. For example, Commons, a media library app offering public domain and freely licensed content, features a page that pushes newly published items in an infinite list. As shown in Fig. 6.(b), when the app is first opened, only the title of the first media item is visible due to screen size limitations, so PBFDRDROID+ determines that the list contains a single item. If the user then clicks on the image of the second media item and returns from the detail page, the second item is centered on the screen while the title of the first item remains visible. This leads PBFDRDROID+ to believe the number of list items increased from one to two, and it wrongly identifies the sequence as an CREATE functionality. Fortunately, such cases are rare—only Commons and TikTok among all tested apps are affected—and they can be avoided by excluding such screens from being used as the start or end state of a DMF. (3) *Bug-induced misidentification (3/12 cases)*. Some identifications occurred under buggy app states, where the captured event sequence could not successfully perform the intended functionality in a correct execution context. Such misidentifications can often reveal real bugs in the app and may be valuable when discovered through manual inspection. For example, when renaming a system folder on a mobile device using Amaze, the app does not show any warning indicating that the folder cannot be renamed. Instead, it keeps the original folder and creates a new one with the intended new name. As a result, PBFDRDROID+ mistakenly identifies this sequence as a CREATE functionality, while it is actually a UPDATE functionality. Consequently, the synthesized property records only the addition of the new folder, but fails to capture that the original folder should have been replaced. Although this property is misidentified, it can still help expose a real bug in the app: once the property is executed, the recorded data model becomes inconsistent with the actual app data. Later, during search property execution, if PBFDRDROID+ selects the

TABLE IV
BUG FINDING RESULTS OF PBFDRDROID+.

App Name	Issue ID	Issue State	Related DMFs	#Steps	Consequence	Description
Markor	#1652	Fixed	CREATE,SEARCH	11	Wrong behavior	No files can be searched in the root directory
	#1668	Fixed	CREATE,UPDATE	8	Wrong behavior	Renaming will fail if new name contains ""?
	#1695	Fixed	CREATE,UPDATE,SEARCH	8	Data loss	Renaming will overwrite the same case sensitive name files
	#1698	Fixed	CREATE,SEARCH	6	Crash	Rotating the screen after searching causes a crash
	#1699	Fixed	CREATE,UPDATE	6	Crash	Rotating the screen while editing will cause a crash
Aard2	#140	Fixed	CREATE,SEARCH	7	Wrong behavior	Symbols in search text are ignored when searching
SimpleTask	#1156	Confirmed	CREATE,SEARCH	9	Wrong behavior	Searching again after canceling a search will not work
SkyTube	#1045	Pending	CREATE,DELETE,SEARCH	9	Wrong behavior	The function of clearing the blocked channel list is unstable
	#1044	Confirmed	CREATE,SEARCH	4	Infinite loading	Refreshing video list after blocking any channel causes infinite loading
AnyMemo	#524	Pending	CREATE,UPDATE	6	Update delay	The card list is not updated in time after editing any card
	#523	Pending	CREATE,SEARCH	12	Data loss	The "Reset All Preferences" option will delete the added card
	#538	Pending	DELETE	7	Wrong behavior	Clicking the delete button fails to remove the category
Amaze	#3207	Fixed	CREATE/UPDATE,SEARCH	9	Infinite loading	Searching for hidden folders causes crashes or infinite loading
	#3298	Confirmed	CREATE/UPDATE,SEARCH	14	Wrong behavior	Renaming will fail on the search results page
	#3357	Confirmed	CREATE/UPDATE,SEARCH	7	Wrong behavior	Search results are not sorted by relevance
AnkiDroid	#10431	Fixed	CREATE,READ	14	Wrong behavior	Cards that use the wrong card template will show up empty
	#11626	Confirmed	CREATE,UPDATE,READ	12	Wrong behavior	Cards cannot be edited when their type is changed to cloze
	#11280	Fixed	CREATE	10	Crash	Saving an empty video in card causes a crash
Wikipedia	#T305555	Fixed	CREATE,UPDATE,SEARCH	11	Update delay	Cannot search the favorites by new name after renaming the favorites
Tasks	#1869	Confirmed	CREATE,READ	7	Wrong behavior	Tasks can be filtered by other criteria but not by date
RadioDroid	#1099	Pending	-	4	Crash	Long-pressing the radio information in the history causes a crash
ActivityDiary	#310	Fixed	SEARCH	6	Crash	Screen rotation after entering invalid date in search bar cause a crash
	#311	Pending	CREATE,DELETE	11	Crash	Deleted activity cannot be recovered correctly
	#320	Pending	SEARCH,DELETE	5	Wrong behavior	The activities in the search list cannot be deleted properly
AndBible	#3455	Confirmed	SEARCH	2	Crash	A crash occurred during search.
Anntennapod	#5863	Fixed	READ	3	Wrong behavior	Clicking "Next" has no effect after the player is paused.
Omin Notes	#886	Confirmed	CREATE,DELETE,SEARCH	14	Wrong behavior	Deleted items can still be searched
	#1014	Pending	CREATE,SEARCH	7	Crash	Adding a reminder to a note via the search screen causes a crash.
Commons	#6390	Confirmed	-	5	Crash	Pressing back after denying storage permission causes a crash
TikTok	-	Fixed	CREATE,READ	11	Wrong behavior	Videos of blocked users can still be seen in the recommendation page
	-	Fixed	CREATE,READ	9	Update delay	The status is not updated in time after unblocking the user
Lark	-	Fixed	CREATE,UPDATE,SEARCH	14	Update delay	Cannot search the folder by new name after renaming the folder

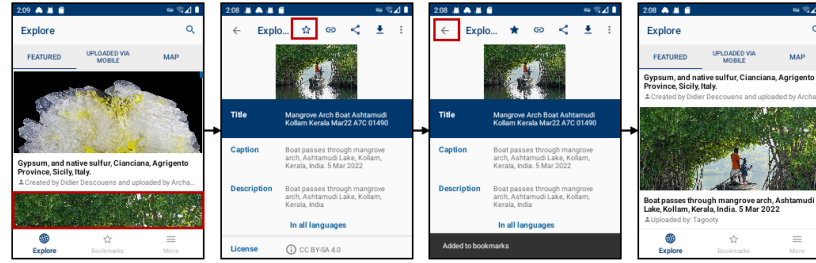


Fig. 6. Illustrative example of incorrect DMF identifications.

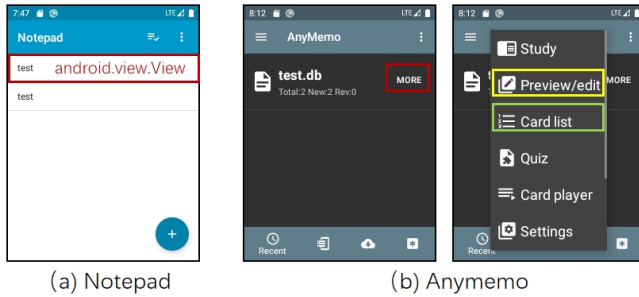


Fig. 7. Illustrative example of missed DMF identifications.

old folder name that should already have been replaced, the postcondition of the search property may be violated, thereby triggering a bug report. At that point, developers inspecting the report may feel that PBFDRDROID+ reported the bug “by accident.” In fact, however, this is not accidental, because it essentially reflects the deviation between the app’s actual erroneous behavior and the expected behavioral characteristics captured by the property.

2) *Analysis of Missed DMF Identifications:* We analyzed all DMFs that were manually identified but missed by PBFDRDROID+, and categorized the reasons for these failures into two types: (1) *Non-standard UI structures* (5/12 cases).

One reason PBFDRDROID+ struggle to identify certain DMFs is that some interface features do not match the expected characteristics of DMFs. For example, Notepad is a note-taking app. as shown in Fig. 7.(a), unlike typical designs, it does not use a list view to display notes. Instead, each note is rendered as an individual `android.view.View`, making it hard to match our defined patterns. Fortunately, such exceptions are rare, with Notepad being the only app we tested that follows this design. (2) *Complex multi-step functionalities* (7/12 cases). Another reason PBFDRDROID+ fail to identify DMFs is due to the complexity of function steps or the deep scenarios in which they occur, making it difficult for the automatic exploration strategy to cover all possibilities. For example, anymemo is a vocabulary learning app. In this app, the add word function and the view word list function are located on two different pages, as shown in Fig. 7.(b). If the impact of the add function is to be observed, the user needs to start from the main page, click the MORE button, then click the “Card list” option (highlighted in red) to check the current word list, then click the back button, click MORE again, select the “Preview/edit” option (highlighted in yellow), complete the add function, click back, and then click MORE again to select the “Card list” option. This sequence allows the newly added word to be observed. Although theoretically there are still possible

paths that trigger the events containing the above steps, for such complex functionalities, we believe manually writing the properties would be more effective.

C. Results for RQ2

1) *Effectiveness of PBFDRDROID+*: Table IV summarizes the bug-finding results, including the app name, the bug ID, the bug state (*fixed*, *confirmed*, or *pending*), related DMFs (which are *necessary* for bug manifestation), the length of minimal bug-reproducing test (in the number of UI events), consequence, and a brief description of the bug. PBFDRDROID+ found 32 unique and previously unknown bugs in 17 out of the 20 tested apps, comprising 23 non-crashing bugs and 9 crashing bugs. To date, 24 bugs have been confirmed, with 15 already fixed, while the remaining are still pending (none of them was rejected). Specifically, 30 of the 32 bugs are DMEs, and 25 DMEs of these require the combination of two or more DMFs for manifestation. Moreover, we received positive feedback from app developers, who commented the found bugs are important and non-trivial. For example, SkyTube [44]'s developer commented “*This bug is really important and annoying*”, and AnkiDroid [12]'s developer commented “*It's a lot more complicated than I thought at first glance*”.

2) *Diversity of bugs found by PBFDRDROID+*: In summary, PBFDRDROID+ found 23 non-crashing bugs of different consequences. We classified them into four types and illustrate some assorted bug samples. (1) *Unexpected wrong behaviors* (15/23 bugs): 14 DMEs lead to unexpected wrong app behaviors. For example, SimpleTask [45], a task management app, has a search function. If a user searches again after canceling a previous search, the user will not be able to find any matching results. (2) *Delayed data update* (4/23 bugs): 4 DMEs lead to delayed data update. For example, in Wikipedia [46], if a user changes the name of the favorite, the user will not be able to search the favorites via the new name for a long time. (3) *User data loss* (2/23 bugs): 2 DMEs lead to severe user data loss. For example, AnyMemo [47], a flashcard learning app, has a creation function for users to add their own cards. However, due to an error in this function, if a user selects the option “reset all preferences” in the app setting, all the user-added cards will be unexpectedly deleted from the database. (4) *Infinite loading* (2/23 bugs): 2 DMEs lead to infinite loading. For example, in Skytube [44], if a user adds a channel to the blocked list and then refreshes the video list, the app will enter into infinite loading and do not respond anymore.

D. Results for RQ3

Experimental results show that PBFDRDROID+ identified a total of 32 DMEs in 48 hours. However, GENIE, ODIN, and MONKEY failed to detect any of the 23 non-crashing bugs uncovered by PBFDRDROID+. Furthermore, among the 9 crashing bugs found by PBFDRDROID+, they were only able to identify 1, 1, and 3, respectively. We further analyzed the results of GENIE and ODIN to understand the reasons why they missed all the non-crashing DMEs. We find two major reasons. First, *their automated oracles are limited in finding DMEs*. GENIE finds non-crashing bugs based on the *independent view*

property, that is interacting with one GUI view should not affect the states of the others and only adds additional GUI effects. However, we find many of the DMEs are not within the scope of this generic oracle. For example, the Amaze's bug in Fig. 1 cannot be detected by GENIE as no independent view exists for Folder “old-name”. ODIN uses a heuristic oracle to find non-crashing bugs, that is appending the same events to the test inputs terminating at similar GUI layouts, and clustering the manifested behaviors to identify the minorities as suspicious bugs. However, one abstraction rules used by Odin ignores the texts displayed on the GUI pages when clustering. As a result, it cannot identify those bugs which lead to incorrect displayed texts. For example, in Fig. 1(o), the folder “old-name” is not correctly renamed, which thus cannot be captured by ODIN. Second, *their randomly generated tests are of low quality*. Many tests generated by GENIE and ODIN did not cover meaningful app functions. As a result, it is difficult for them to explore diverse app states to find DMEs, which may require long and specific event traces (see #Steps in Table IV).

Furthermore, PBFDRDROID+ demonstrated superior performance in terms of false positive rates. After automated deduplication, PBFDRDROID+ generated a total of 41 bug reports, of which only 9 were false positives, resulting in a false positive rate of only 21.9% ($\approx 9/41$). This is significantly lower than the false positive rates of GENIE (38/60 $\approx 63.3\%$) and ODIN (238/358 $\approx 66.5\%$). These results indicate that, compared to generic oracles, the use of abstract property templates and app-specific property synthesis can identify bugs with much higher precision while substantially reducing manual effort. Overall, these findings suggest that PBFDRDROID+ serves as a vital complement to state-of-the-art techniques in detecting DMEs. These results show that PBFDRDROID+ can complement the state-of-the-art techniques in finding DMEs.

E. Results for RQ4

Table V compares Baseline A, B and PBFDRDROID+ in the numbers of found bugs. Baseline A and B only found 1 and 15 bugs, respectively. Moreover, *all* the bugs found by Baseline A and B were found by PBFDRDROID+. Thus, PBFDRDROID+ is much more effective than Baseline A and B. *It indicates that interleaving different DMFs and other possible events is crucial for improving the bug finding ability*.

Baseline B is limited due to two major reasons. First, most of the DMEs (78.1% $\approx 25/32$) requires combining two or more DMFs for bug manifestation. When interleaving one single DMF with other possible events, Baseline B may not be able to cover other DMFs by random exploration. For example, to find the bug of “Rename folder” in Fig. 1, Baseline B has to create a folder “old-name”, switch to the up-level directory and search folder “old-name” via random exploration, which is difficult. In our experiment, Baseline B only finds 6 bugs which require two DMFs. Second, because Baseline B only considers one single DMF, it cannot simulate the data changes when other DMFs are covered by chance. For example, if Baseline B tests a SEARCH DMF, the data added by CREATE cannot be checked because that data will not be

TABLE V
COMPARISON BETWEEN BASELINE A, BASELINE B, AND PBFDRDROID+, .
"C" AND "NC" REPRESENT CRASH AND NON-CRASHING BUGS,
RESPECTIVELY.

Tool Failure Type #Bugs	Baseline A		Baseline B		PBFDRDROID+	
	C	NC	C	NC	C	NC
	0	1	9	6	9	19

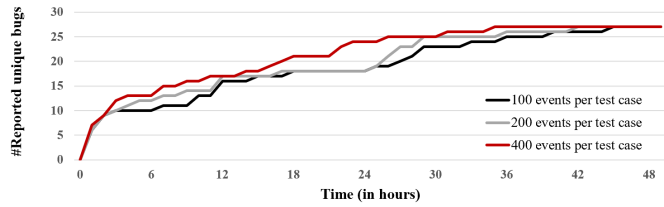


Fig. 8. Number of found unique bugs under different L_{max} .

recorded. Baseline A is further limited because most DMFs only fail in some corner cases when running independently. However, Baseline A cannot reach such cases due to the lack of some random events. For example, AnkiDroid's card editing function fails only when the default card type is changed (which can only be covered by a random event).

Comparison between different L_{max} s. Fig. 8 compares the tool effectiveness under the configurations of 100, 200 and 400 events per test, which are denoted by the black, grey, and red curves, respectively. The horizontal axis denotes the 48-hour testing time, and the vertical axis denotes the number of found unique bugs. *We can see that the three configurations of L_{max} do not have distinct impacts on tool effectiveness.* When L_{max} is set as 400, PBFDRDROID+ can find bugs more quickly (because it may interleave more DMFs in one GUI test), but at last the three configurations found the same numbers of unique bugs on the 18 open-source apps.

V. DISCUSSION

This section provides an extended discussion of our work in the following three aspects: (1) comparison of manual and automated DMF definitions (2) generability of our approach, and (3) threats to validity.

Generability of our approach. Despite our approach focusing on finding DMEs. This property-based fuzzing (PBF) approach has broader applicability. When the property $\phi = \langle Pre, E, Post \rangle$ is instantiated on other app functionalities, PBFDRDROID+ can help automatically find the other types of bugs in Android apps. For example, if we define the property of some registration functionality in the app, our approach can check whether the registration can always succeed. In the future, we will explore how to extend property-based testing for more generic app properties [28].

Threats to validity. The first threat is the representativeness of app subjects. To this end, our study includes different categories of open-source and industrial apps, many of which are popular on GitHub and Google Play. Thus, they can represent real-world apps. Another potential threat is the general prevalence of DMEs in the Android ecosystem. Our preliminary investigation on well-maintained and popular apps indicates that DMEs constitute a considerable portion of non-crashing bugs. While this supports the applicability of our

approach, future work could extend similar ideas to detect other categories of non-crashing bugs, further enhancing the generality and impact of the technique. A further threat lies in the choice of testing configurations, which could potentially affect the bug detection results. Our experiments were conducted under specific testing configurations, such as the allocated exploration time, selected to be both practical for large-scale testing and sufficient to trigger diverse app behaviors. We also verified that varying these parameters within reasonable ranges does not materially change the key trends in our results, which helps mitigate this threat. Finally, although our fuzzing strategy uses random interleaving of events, it has already proven effective in uncovering diverse app states and detecting bugs. Future enhancements, such as incorporating lightweight static analysis or model-based guidance, may further improve coverage without undermining automation.

Possibility of LLM-assisted Property Refinement. Although our experimental results show that the rule-based redundancy removal and precondition refinement methods are effective and generalizable, and property generation is essentially a one-time cost in practice (making manual refinement acceptable if needed), we still considered an alternative approach to address future scenarios where mobile apps may become more complex and require new rules. Specifically, we conducted a preliminary experiment to evaluate whether Large Language Models (LLMs) can correctly remove redundancies and generate preconditions when refinement rules are described in natural language. The results indicate that LLMs can handle these simple refinement tasks with reasonable accuracy (96.5%). This suggests that LLMs can serve as a practical extension mechanism when new app-specific refinement patterns emerge and manual encoding becomes inconvenient. In this sense, LLMs provide a flexible alternative for scaling property refinement to more complex mobile applications. Nonetheless, considering the additional overhead introduced by LLMs, we did not integrate them into our open-source tool. Instead, our released implementation employs explicitly encoded rules for property deduplication and precondition refinement, which is sufficient for current mobile apps while avoiding extra computational and engineering costs. For future use, we have also released the prompts used for LLM-assisted refinement in our open-source data, allowing researchers or users to use or adjust them as needed.

Analysis of Manual and Automated DMF Construction and Refinement. In the earlier version, DMFs were manually defined by selecting a major data type and specifying CREATE, READ, UPDATE, DELETE, and SEARCH operations. In the current version, PBFDRDROID+ automatically synthesizes DMFs, removing the need for manual definitions. Beyond the experimental results presented earlier, we further analyze the impact of manual and automated DMF definitions on DME detection. Only 2 out of the 27 DMEs detected by manual DMF definitions cannot be found by PBFDRDROID+ (Amaze #4330 and Notepad #134). In addition, PBFDRDROID+ identifies 61 extra DMFs, from which we retain those providing unique functionality, enabling detection of 4 additional DMEs. We attribute this observation to the fact that the core functionalities of most apps rely on only a small number of DMFs, suggesting

that increasing the number of DMFs does not necessarily lead to a significant increase in DME detections.

Moreover, to balance usability and accuracy, PBFDRDROID+ supports two usage modes: a manual refinement mode and a fully automatic mode. We conduct a user study to measure the human effort required by these two modes and summarize their respective trade-offs to guide users in selecting an appropriate usage strategy. (1) In the *manual refinement mode*, testers refine synthesized properties to improve preconditions and reduce false positives. According to our user study with five graduate student participants (non-authors) who have basic knowledge of properties and their specification, refining properties for 20 apps takes 46 minutes in total on average (approximately 138 seconds per app), which is substantially lower than fully manual property definition (about 780 seconds per app in the conference version). (2) In the *fully automatic mode*, synthesized properties are used directly without modification, requiring no manual effort. Experimental results (RQ3) show that even without refinement, the false positive rate remains lower than existing automated functional bug detection methods. Moreover, a separate user study with five graduate student participants (non-authors) without prior knowledge of properties shows that checking each report takes only 48 seconds on average, with no more than 112 seconds spent on false positives per app per testing round. Overall, when domain knowledge is available, manual specification is suitable for precise verification, while automated synthesis enables broad validation at a lower cost. These two modes present a trade-off between human effort and accuracy, and can be selected flexibly depending on the application scenario.

VI. RELATED WORK

Finding non-crashing bugs for Android apps. Most automated GUI testing tools [17], [18], [19], [20], [21], [22], [23], [24], [25], [26] use runtime exceptions as the oracle [48], [49]. Therefore, they *cannot* find the non-crashing DMEs which our work focuses on. To tackle the oracle problem, three major categories of testing techniques have been proposed. Table VI compares the representative work in these categories, which we explain as follows. The first category of prior work [29], [37], [50], [41], [51], [52] adopts metamorphic relations to generate automated oracles. However, most of these work targets specific types of non-crashing bugs (*e.g.*, system setting related defects [37], [53], data loss issues [50], [41]). Only GENIE [29] targets *generic* non-crashing bugs, but it missed all the non-crashing DMEs due to the limitation of its metamorphic relation (discussed in Section IV-C). The second category uses differential analysis [30] or testing [54], [43]. ODIN [30] finds non-crashing bugs by differing the buggy and normal app behaviors based on the heuristic oracle “bugs as deviant behaviors” [31]. But it missed all the non-crashing DMEs due to the limitation of its abstraction rule (discussed in Section IV-C). Some work [43], [54] uses differential testing to find (non-crashing) compatibility bugs based on different devices. They *cannot* find DMEs. The third category focuses on testing based on given test inputs. This includes approaches that migrate or synthesize tests from

TABLE VI
DIFFERENCES BETWEEN EXISTING TESTING TOOLS AND OURS IN FINDING NON-CRASHING BUGS OF ANDROID APPS.

Tool	What to provide before testing?	Types of test oracle
GENIE	-	Metamorphic relations
ODIN	-	Implicit knowledge
APPFLOW	Modular tests	Pseudo-oracles
CHIMPCHECK	Example-based Tests	Assertions
PBFDRDROID+	-	Model-based specification

one app to another with similar functionalities [55], [56], [57], [58], [59], [60]. APPFLOW [55] synthesizes from the modular (abstract) tests. However, these work does not target DMEs. The synthesis accuracy is not high (which may lead to many false positives) [61]. In contrast, PBFDRDROID+ can automatically generate GUI tests without false positives. Some recent studies [62], [63], [64] apply property-based testing to Android apps, but they also rely on user-provided test inputs. Specifically, CHIMPCHECK [62] requires example-based tests, KEA [63] depends on properties written in a domain-specific language, and PDTRDROID [64] needs privacy specifications written in natural language. In contrast, PBFDRDROID+ can automatically test any app by synthesizing properties from general-purpose templates, without requiring manual input. Second, due to the limited expressiveness of these techniques, their assertions cannot effectively capture data changes when different DMFs are interleaved. As a result, if adapted to our setting—by manually providing the required test inputs for each DMF—their capability would be comparable to Baseline B in our evaluation (see Section IV-E).

Property-based testing and model-based testing. Property-based testing (PBT) was popularized by Claessen and Hughes [65]. Specifically, the two key elements of PBT are (1) the data generators *and* (2) the property specifications. In our work, the data generator is designed to generate random UI-based event traces (by interleaving the DMFs and other events), and the properties are specified in the model-based properties [66] (an abstract model capturing data update effect of the DMFs in the apps). Our work is different from the prior PBT work in two aspects. First, the prior work designs the data generators for their own specific domains [67], [68], [69], [70], [71], [68], [72], which cannot be applied for Android apps. Second, these prior work specifies the properties in the *assertions* [67], [68], [69], [70], [71] or temporal logic formula [68], [72], which are not suitable to capture the data update effect in our setting. To our knowledge, few work in the literature combines PBT with the model-based properties [66].

On the other hand, using an abstract model as the specification is the foundation of model-based testing (MBT) [73]. A lot of work [74], [75], [19], [76], [20], [77] exists in applying MBT to test Android apps. However, the models in these work are the finite state machine based UI models (in which each state is an abstract UI layout and each edge is a UI event), which are used to guide test generation rather than deriving the oracles. These models are different from the model in our work. Therefore, all of these MBT work [74], [75], [19], [76], [20] cannot find non-crashing bugs.

Finding the errors related to CRUD. Our work finds software errors related to the CRUD operations. Costa *et al.* [2] model the “Find” operation (*i.e.*, SEARCH) in Android apps

as one UI test pattern to check its correctness. Mariani *et al.* [3] specify the CRUD operations of Web apps in ALLOY specification language and build a tool AUGUSTO to generate semantic tests with oracles. However, AUGUSTO *only* tests one CRUD operation independently, and does not consider their combinations. Since a direct comparison is infeasible (as AUGUSTO targets Web apps), Baseline A in our evaluation (see Section IV-E) can be viewed as a similar implementation of AUGUSTO. Our approach which interleaves different DMFs of CRUD operations is shown to be much more effective than Baseline A. Some work [78], [79] uses metamorphic testing to find the CRUD errors in database management systems.

VII. CONCLUSION

We introduce an automated property-based fuzzing approach to effectively finding DMEs, which combines the idea of property-based testing and model-based properties. The realization of our idea, PBFDRDROID+, has successfully discovered 32 previously unknown bugs from 20 popular Android apps. Among these bugs, 23 are non-crashing DMEs, which cannot be found by the state-of-the-arts. So far, 24 have been confirmed and 15 fixed by the developers. The majority (20 out of 25) cause non-crashing failures and are hard to be detected by state-of-the-art automatic testing tools.

VIII. ACKNOWLEDGMENTS

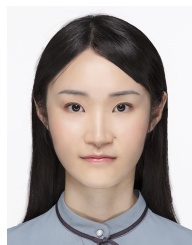
We thank the anonymous TSE reviewers for their valuable feedback. The work is supported by the NSFC Project (Grant No.62502077, 62572192, 92582203 and 62072178) and the Fundamental Research Funds for the Central Universities (ZYGX2024XJ036).

REFERENCES

- [1] Business of Apps. (2022) Android statistics (2022). Retrieved 2025-8 from <https://www.businessofapps.com/data/android-statistics/>.
- [2] P. Costa, A. C. Paiva, and M. Nabuco, "Pattern based gui testing for mobile applications," in *Proceedings of the 9th International Conference on the Quality of Information and Communications Technology (QUATIC)*, 2014, pp. 66–74.
- [3] L. Mariani, M. Pezzè, and D. Zuddas, "Augusto: Exploiting Popular Functionalities for the Generation of Semantic GUI Tests with Oracles," in *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, 2018, pp. 280–290.
- [4] Wikipedia. (2022) Create, read, update and delete. Retrieved 2025-8 from https://en.wikipedia.org/wiki/Create,_read,_update_and_delete.
- [5] S. R. Choudhary, A. Gorla, and A. Orso, "Automated test input generation for android: Are we there yet? (E)," in *30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2015, pp. 429–440.
- [6] W. Wang, D. Li, W. Yang, Y. Cao, Z. Zhang, Y. Deng, and T. Xie, "An empirical study of android test generation tools in industrial cases," in *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE)*, 2018, pp. 738–748.
- [7] T. Su, J. Wang, and Z. Su, "Benchmarking automated GUI testing for android against real-world bugs," in *29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2021, pp. 119–130.
- [8] Benoît Joossen. (2018) Voice memos "epidemic failure" and how to avoid it. Retrieved 2025-8 from <https://aeroquartet.com/wordpress/2019/03/05/voice-memos-epidemic-failure-and-how-to-avoid-it/>.
- [9] Hacker News. (2013) Tell facebook: There's a severe bug when changing profile pics on the ios app. Retrieved 2025-8 from <https://news.ycombinator.com/item?id=6456285>.
- [10] Sixth Tone. (2019) E-Commerce App Loses 'Tens of Millions' From Coupon Glitches. Retrieved 2025-8 from <https://www.sixthtone.com/news/1003483/e-commerce-app-loses-tens-of-millions-from-coupon-glitches>.
- [11] A. Team. (2022) AmazeFileManager. Retrieved 2025-8 from <https://github.com/TeamAmaze/AmazeFileManager>.
- [12] AnkiDroid Team. (2022) AnkiDroid. Retrieved 2025-8 from <https://github.com/ankidroid/Anki-Android>.
- [13] k-9 Team. (2022) k-9. Retrieved 2025-8 from <https://github.com/thundernest/k-9>.
- [14] WordPress Team. (2022) Wordpress. Retrieved 2025-8 from <https://github.com/wordpress-mobile/WordPress-Android>.
- [15] P. Kong, L. Li, J. Gao, K. Liu, T. F. Bissyandé, and J. Klein, "Automated testing of android apps: A systematic literature review," *IEEE Trans. Reliability*, vol. 68, no. 1, pp. 45–66, 2019.
- [16] P. Tramontana, D. Amalfitano, N. Amatiucci, and A. R. Fasolino, "Automated functional testing of mobile applications: a systematic mapping study," *Software Quality Journal*, vol. 27, no. 1, pp. 149–201, 2019.
- [17] Monkey Team. (2022) Android Monkey. Retrieved 2025-8 from <https://developer.android.com/studio/test/monkey>.
- [18] K. Mao, M. Harman, and Y. Jia, "Sapienz: multi-objective automated testing for android applications," in *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA)*, 2016, pp. 94–105.
- [19] T. Su, G. Meng, Y. Chen, K. Wu, W. Yang, Y. Yao, G. Pu, Y. Liu, and Z. Su, "Guided, stochastic model-based gui testing of android apps," in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, 2017, pp. 245–256.
- [20] T. Gu, C. Sun, X. Ma, C. Cao, C. Xu, Y. Yao, Q. Zhang, J. Lu, and Z. Su, "Practical gui testing of android applications via model abstraction and refinement," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, 2019, pp. 269–280.
- [21] J. Wang, Y. Jiang, C. Xu, C. Cao, X. Ma, and J. Lu, "Combodroid: Generating high-quality test inputs for android apps via use case combinations," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE)*, 2020, p. 469–480.
- [22] Z. Dong, M. Böhme, L. Cojocar, and A. Roychoudhury, "Time-travel testing of android apps," in *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*, 2020, pp. 1–12.
- [23] Z. Lv, C. Peng, Z. Zhang, T. Su, K. Liu, and P. Yang, "Fastbot2: Reusable automated model-based GUI testing for android enhanced by reinforcement learning," in *37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2022, pp. 135:1–135:5.
- [24] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, X. Che, D. Wang, and Q. Wang, "Make llm a testing expert: Bringing human-like interaction to mobile gui testing via functionality-aware decisions," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
- [25] H. Hu, H. Wang, R. Dong, X. Chen, and C. Chen, "Enhancing gui exploration coverage of android apps with deep link-integrated monkey," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 6, pp. 1–31, 2024.
- [26] Y. Zhao, B. Harrison, and T. Yu, "Dinodroid: Testing android apps using deep q-networks," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 5, pp. 1–24, 2024.
- [27] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE transactions on software engineering (TSE)*, pp. 507–525, 2014.
- [28] Y. Xiong, M. Xu, T. Su, J. Sun, J. Wang, H. Wen, G. Pu, J. He, and Z. Su, "An empirical study of functional bugs in android apps," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2023, pp. 1319–1331.
- [29] T. Su, Y. Yan, J. Wang, J. Sun, Y. Xiong, G. Pu, K. Wang, and Z. Su, "Fully automated functional fuzzing of android apps for detecting non-crashing logic bugs," *Proceedings of the ACM on Programming Languages (OOPSLA)*, pp. 1–31, 2021.

- [30] J. Wang, Y. Jiang, T. Su, S. Li, C. Xu, J. Lu, and Z. Su, "Detecting non-crashing functional bugs in android apps via deep-state differential analysis," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2022, pp. 434–446.
- [31] D. R. Engler, D. Y. Chen, and A. Chou, "Bugs as deviant behavior: A general approach to inferring errors in systems code," in *Proceedings of the 18th ACM Symposium on Operating System Principles (SOSP 2001)*, 2001, pp. 57–72.
- [32] T. Y. Chen, S. C. Cheung, and S. M. Yiu, "Metamorphic testing: a new approach for generating next test cases," HKUST-CS98-01, Hong Kong University of Science and Technology, Tech. Rep., 2020.
- [33] K. Claessen and J. Hughes, "QuickCheck: a lightweight tool for random testing of Haskell programs," in *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP)*, 2000, pp. 268–279.
- [34] GeeksforGeeks. (2020) MVC (Model View Controller) Architecture Pattern in Android. Retrieved 2025-8 from <https://www.geeksforgeeks.org/mvc-model-view-controller-architecture-pattern-in-android-with-example/>.
- [35] C. Pacheco, S. K. Lahiri, M. D. Ernst, and T. Ball, "Feedback-directed random test generation," in *29th International Conference on Software Engineering (ICSE)*, 2007, pp. 75–84.
- [36] uiautomator2 Team. (2022) uiautomator2. Retrieved 2025-8 from <https://github.com/openatx/uiautomator2>.
- [37] J. Sun, T. Su, J. Li, Z. Dong, G. Pu, T. Xie, and Z. Su, "Understanding and finding system setting-related defects in android apps," in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2021, pp. 204–215.
- [38] Tiktok Team. (2021) Tiktok. Retrieved 2025-8 from <https://www.tiktok.com>.
- [39] Lark Team. (2021) Lark. Retrieved 2025-8 from <https://www.larksuite.com/>.
- [40] Google. (2021) Google Play. Retrieved 2025-8 from <https://play.google.com/store>.
- [41] O. Riganelli, S. P. Mottadelli, C. Rota, D. Micucci, and L. Mariani, "Data loss detector: automatically revealing data loss bugs in android apps," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2020, pp. 141–152.
- [42] W. Guo, Z. Dong, L. Shen, W. Tian, T. Su, and X. Peng, "Detecting and fixing data loss issues in android apps," in *ISSTA '22: 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, Virtual Event, South Korea, July 18 - 22, 2022*, 2022, pp. 605–616.
- [43] M. Fazzini and A. Orso, "Automated cross-platform inconsistency detection for mobile apps," in *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2017, pp. 308–318.
- [44] SkyTube Team. (2022) Skytube. Retrieved 2025-8 from <https://github.com/SkyTubeTeam/SkyTube>.
- [45] SimpleTask Team. (2022) Simpletask. Retrieved 2025-8 from <https://github.com/mpcjanssen/simpletask-android>.
- [46] Wikipedia Team. (2022) Wikipedia. Retrieved 2025-8 from <https://github.com/wikimedia/apps-android-wikipedia>.
- [47] AnyMemo Team. (2022) Anymemo. Retrieved 2025-8 from <https://github.com/helloworld1/AnyMemo>.
- [48] L. Fan, T. Su, S. Chen, G. Meng, Y. Liu, L. Xu, G. Pu, and Z. Su, "Large-scale analysis of framework-specific exceptions in android apps," in *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, M. Chaudron, I. Crnkovic, M. Chechik, and M. Harman, Eds., 2018, pp. 408–419.
- [49] T. Su, L. Fan, S. Chen, Y. Liu, L. Xu, G. Pu, and Z. Su, "Why my app crashes? understanding and benchmarking framework-specific exceptions of android apps," *IEEE Trans. Software Eng.*, vol. 48, no. 4, pp. 1115–1137, 2022.
- [50] W. Guo, Z. Dong, L. Shen, W. Tian, T. Su, and X. Peng, "Detecting and fixing data loss issues in android apps," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2022, pp. 605–616.
- [51] C. Q. Adamsen, G. Mezzetti, and A. Møller, "Systematic execution of android test suites in adverse conditions," in *Proceedings of the 2015 International Symposium on Software Testing and Analysis (ISSTA)*, 2015, pp. 83–93.
- [52] R. N. Zaeem, M. R. Prasad, and S. Khurshid, "Automated generation of oracles for testing user-interaction features of mobile apps," in *Proceedings of the International Conference on Software Testing, Verification and Validation (ICST)*, 2014, pp. 183–192.
- [53] J. Sun, T. Su, K. Liu, C. Peng, Z. Zhang, G. Pu, T. Xie, and Z. Su, "Characterizing and finding system setting-related defects in android apps," *IEEE Trans. Software Eng.*, vol. 49, no. 4, pp. 2941–2963, 2023.
- [54] Y. Lin, J. F. Rojas, E. T. Chu, and Y. Lai, "On the accuracy, efficiency, and reusability of automated test oracles for android devices," *IEEE Trans. Software Eng.*, pp. 957–970, 2014.
- [55] G. Hu, L. Zhu, and J. Yang, "Appflow: using machine learning to synthesize robust, reusable ui tests," in *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2018, pp. 269–282.
- [56] F. Behrang and A. Orso, "Test migration between mobile apps with similar functionality," in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2019, pp. 54–65.
- [57] J.-W. Lin, R. Jabbarvand, and S. Malek, "Test transfer across mobile apps through semantic mapping," in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2019, pp. 42–53.
- [58] A. Rosenfeld, O. Kardashov, and O. Zang, "Automation of android applications functional testing using machine learning activities classification," in *Proceedings of the 5th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*, 2018, pp. 122–132.
- [59] Y. Zhang, Q. Zhu, J. Yan, C. Liu, W. Zhang, Y. Zhao, D. Hao, and L. Zhang, "Synthesis-based enhancement for gui test case migration," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 869–881.
- [60] B. Beyzaei, S. Talebipour, G. Rafiei, N. Medvidović, and S. Malek, "Automated test transfer across android apps using large language models," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 2227–2250, 2025.
- [61] Y. Zhao, J. Chen, A. Sejfia, M. S. Laser, J. Zhang, F. Sarro, M. Harman, and N. Medvidovic, "Fruiter: a framework for evaluating UI test reuse," in *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2020, pp. 1190–1201.
- [62] E. S. Lam, P. Zhang, and B.-Y. E. Chang, "Chimpcheck: property-based randomized test generation for interactive apps," in *Proceedings of the 2017 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, 2017, pp. 58–77.
- [63] Y. Xiong, T. Su, J. Wang, J. Sun, G. Pu, and Z. Su, "General and practical property-based testing for android apps," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 53–64.
- [64] J. Sun, T. Su, J. Sun, J. Li, M. Wang, and G. Pu, "Property-based testing for validating user privacy-related functionalities in social media apps," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024, pp. 440–451.
- [65] K. Claessen and J. Hughes, "QuickCheck: a lightweight tool for random testing of Haskell programs," in *Proceedings of the fifth ACM SIGPLAN international conference on Functional programming (ICFP)*, 2000, pp. 268–279.
- [66] J. Hughes, "How to specify it!" in *Trends in Functional Programming*. Springer International Publishing, 2020, pp. 58–83.
- [67] S. Karlsson, A. Čaušević, and D. Sundmark, "QuickREST: Property-based Test Generation of OpenAPI-Described RESTful APIs," in *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*, 2020, pp. 131–141.
- [68] A. Santos, A. Cunha, and N. Macedo, "Property-based testing for the robot operating system," in *Proceedings of the 9th ACM SIGSOFT International Workshop on Automating TEST Case Design, Selection, and Evaluation*, 2018, pp. 56–62.
- [69] R. Padhye, C. Lemieux, and K. Sen, "JQF: coverage-guided property-based testing in java," in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2019, pp. 398–401.

- [70] S. Padhiyar and K. Sivaramakrishnan, "Confuzz: Coverage-guided property fuzzing for event-driven programs," in *Practical Aspects of Declarative Languages: 23rd International Symposium, PADL 2021, Copenhagen, Denmark, January 18-19, 2021, Proceedings 23*, 2021, pp. 127–144.
- [71] L. Lampropoulos, M. Hicks, and B. C. Pierce, "Coverage guided, property based testing," *Proc. ACM Program. Lang.*, no. OOPSLA, pp. 181:1–181:29, 2019.
- [72] L. O'Connor and O. Wickström, "Quickstrom: property-based acceptance testing with LTL specifications," in *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, 2022, pp. 1025–1038.
- [73] M. Broy, B. Jonsson, J.-P. Katoen, M. Leucker, and A. Pretschner, *Model-based testing of reactive systems: advanced lectures*. Springer Science & Business Media, 2005.
- [74] D. Amalfitano, A. R. Fasolino, P. Tramontana, B. D. Ta, and A. M. Memon, "Mobiguitar: Automated model-based testing of mobile apps," *IEEE Software*, pp. 53–59, 2015.
- [75] Y.-M. Baek and D.-H. Bae, "Automated model-based android gui testing using multi-level gui comparison criteria," in *2016 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2016, pp. 238–249.
- [76] Y. Li, Z. Yang, Y. Guo, and X. Chen, "Droidbot: a lightweight ui-guided test input generator for android," in *Proceedings of the 2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*, 2017, pp. 23–26.
- [77] S. Karlsson, A. Causevic, D. Sundmark, and M. Larsson, "Model-based automated testing of mobile applications: An industrial case study," in *14th IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW 2021)*, 2021, pp. 130–137.
- [78] M. Rigger and Z. Su, "Testing database engines via pivoted query synthesis," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2020, pp. 667–682.
- [79] —, "Finding bugs in database systems via query partitioning," *Proc. ACM Program. Lang.*, no. OOPSLA, pp. 211:1–211:30, 2020.



Jingling Sun received the B.S. and Ph.D. degrees in Software Engineering from East China Normal University, Shanghai, China, in 2014 and 2023, respectively. She was a visiting researcher at Singapore Management University, Singapore, from 2022 to 2023. She is now a Lecturer at the School of Information and Software Engineering, University of Electronic Science and Technology of China (UESTC), Chengdu, China. Her research focuses on software analysis and testing. For more information, please visit <https://jinglingsun.github.io/>.



Ting Su received the B.S. degree in Software Engineering and the Ph.D. degree in Computer Science from Software Engineering Institute, East China Normal University, Shanghai, China, in 2011 and 2016, respectively. He worked as a postdoc at ETH Zurich, Switzerland, and Nanyang Technological University, Singapore, successively. He is currently a Professor at East China Normal University. His research focuses on software analysis, testing and verification. He has published extensively in top-tier programming language, software engineering and security conferences and journals. For more information, please visit <https://tingsu.github.io/>.



Xixian Liang received the M.S. degree in Software Science and Technology from East China Normal University, China, in 2026, and B.S. degree in Automation Science and Engineering from South China University of Technology, China, in 2023. He is currently a software engineer with Tencent WeChat. His research focuses on automated UI testing. For more information, please visit <https://xixianliang.github.io/resume/>.



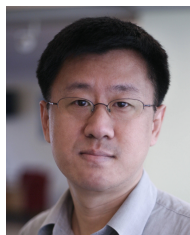
[//cv.juewang.info/](https://cv.juewang.info/).

Jue Wang received his B.S. degree and Ph.D. degree in Computer Science from Nanjing University, Nanjing, China, in 2016 and 2022, respectively. He works as a Senior Research Engineer at ByteDance. His research focuses on software analysis, automated software testing, and LLM-assisted software development, and received two ACM SIGSOFT distinguished paper awards. He has published in top-tier programming languages and software engineering venues, including ICSE, OOPSLA, FSE, ASE, TSE, and ISSTA. For more information, please visit <https://cv.juewang.info/>.



etc). For more information, please visit <https://ggpu-ecnu.github.io/>.

Geguang Pu received the BS degree in mathematics from Wuhan University, in 2000, and the PhD degree in mathematics from Peking University, in 2005. He is a professor with the School of Software Engineering, East China Normal University. His research interests include program testing, analysis and verification. He served as PC members for international conferences such as SEFM, ATVA, TASE etc. He has published more than 70 publications on the topics of software engineering and system verification (including ICSE, FSE, IJCAI, FM, ECAI, CONCUR etc). For more information, please visit <https://ggpu-ecnu.github.io/>.



<https://people.inf.ethz.ch/suz/>.

Zhendong Su received the BS degree in computer science and BA degree in mathematics from the University of Texas at Austin, Austin, TX, and the MS and PhD degrees in computer science from the University of California at Berkeley, Berkeley, CA. He is a professor in computer science at ETH Zurich, where he specializes in programming languages and compilers, software engineering, computer security, deep learning and education technologies. He is a Member of the Academia Europaea and a Fellow of the IEEE. For more information, please visit <https://people.inf.ethz.ch/suz/>.